



CS 498: Data Parallelism

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

For $t = 1$ to T (iters) How to run large training batch size?

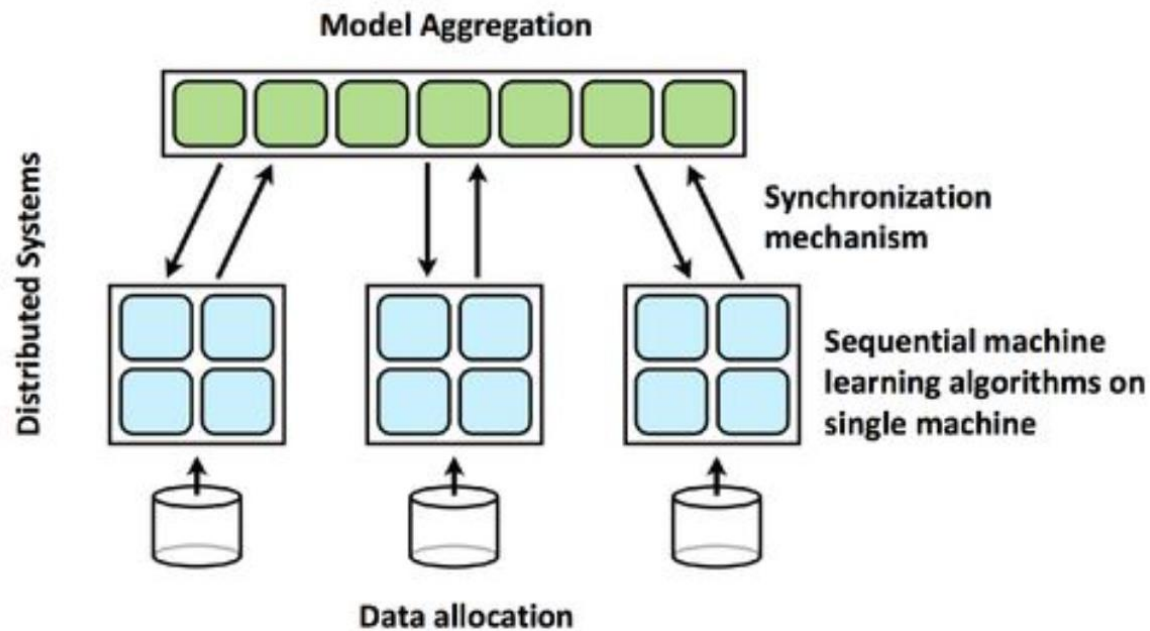
$$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^N \nabla \left(\text{loss}(f_w(x_i, y_i)) \right) \quad // \text{ compute derivative and update}$$

$w -= \Delta w$ // apply update

End

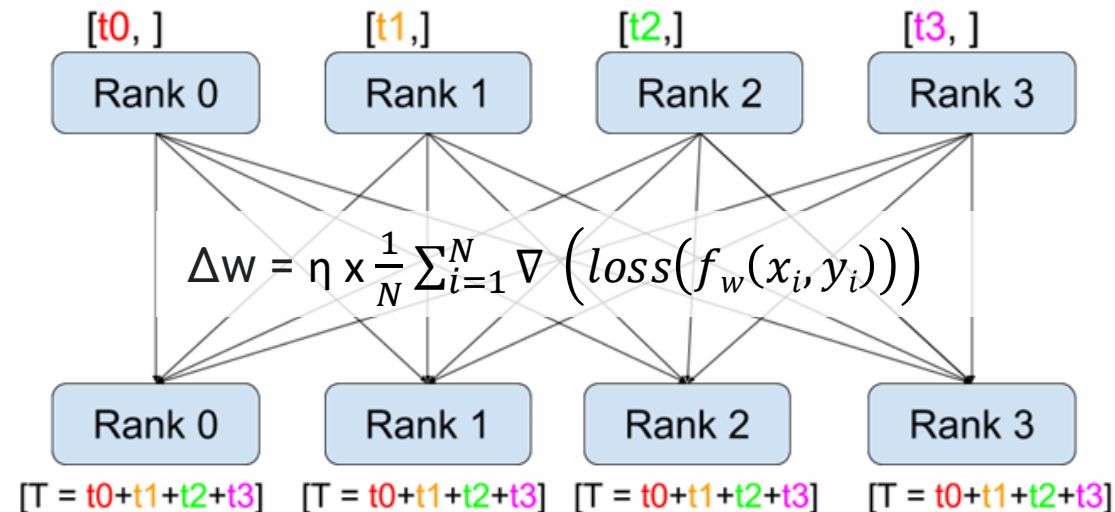
- (Centralized) Parameter Server
- (Decentralized) AllReduce
- Key assumption:
 - The model can fit into a (GPU) worker memory so we can create many model replicas

Recall: Data Parallelism with Parameter Server

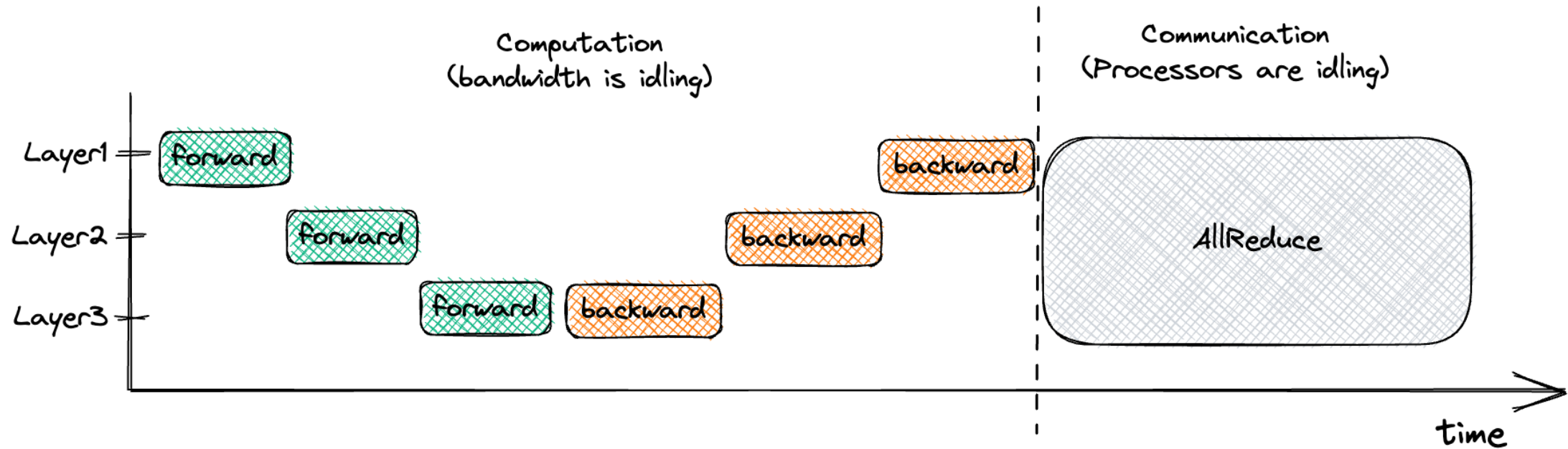


1. Partition the training data
2. Parallel training on different machines
3. Synchronize the local updates
4. Refresh local model with new parameters, then go to 2

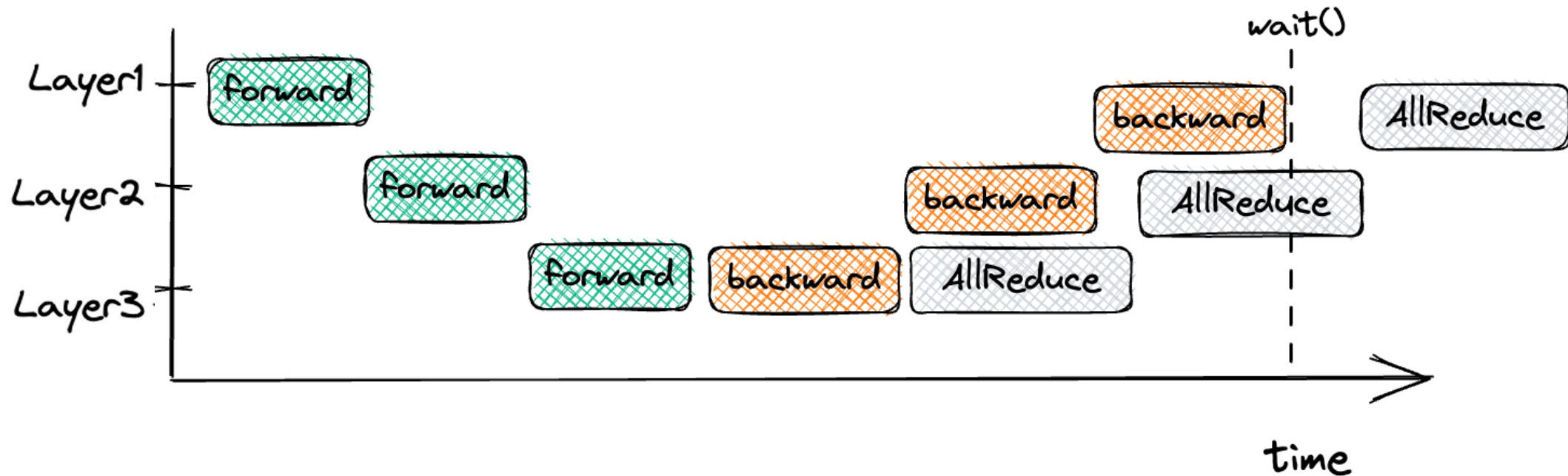
- **Data Parallelism**
 - AllReduce-based DP
 - Communication terminologies



AllReduce based Data Parallelism: Option 1

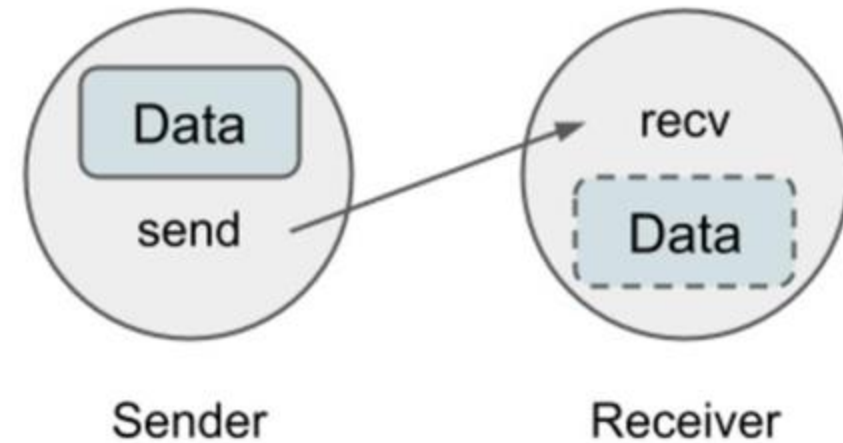
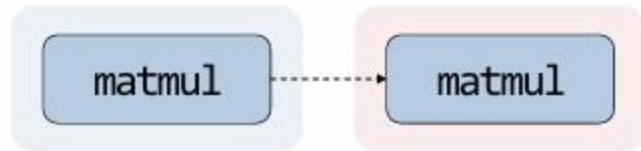


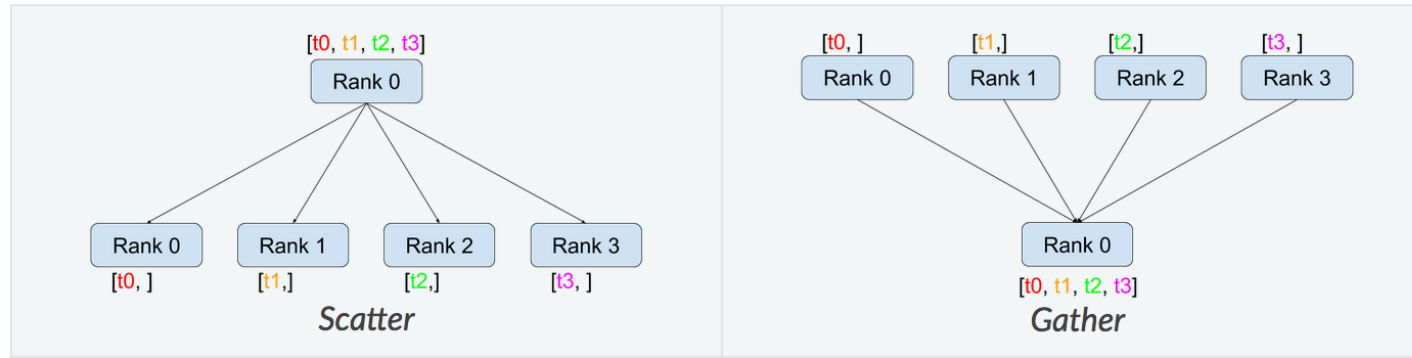
AllReduce based Data Parallelism: Option 2



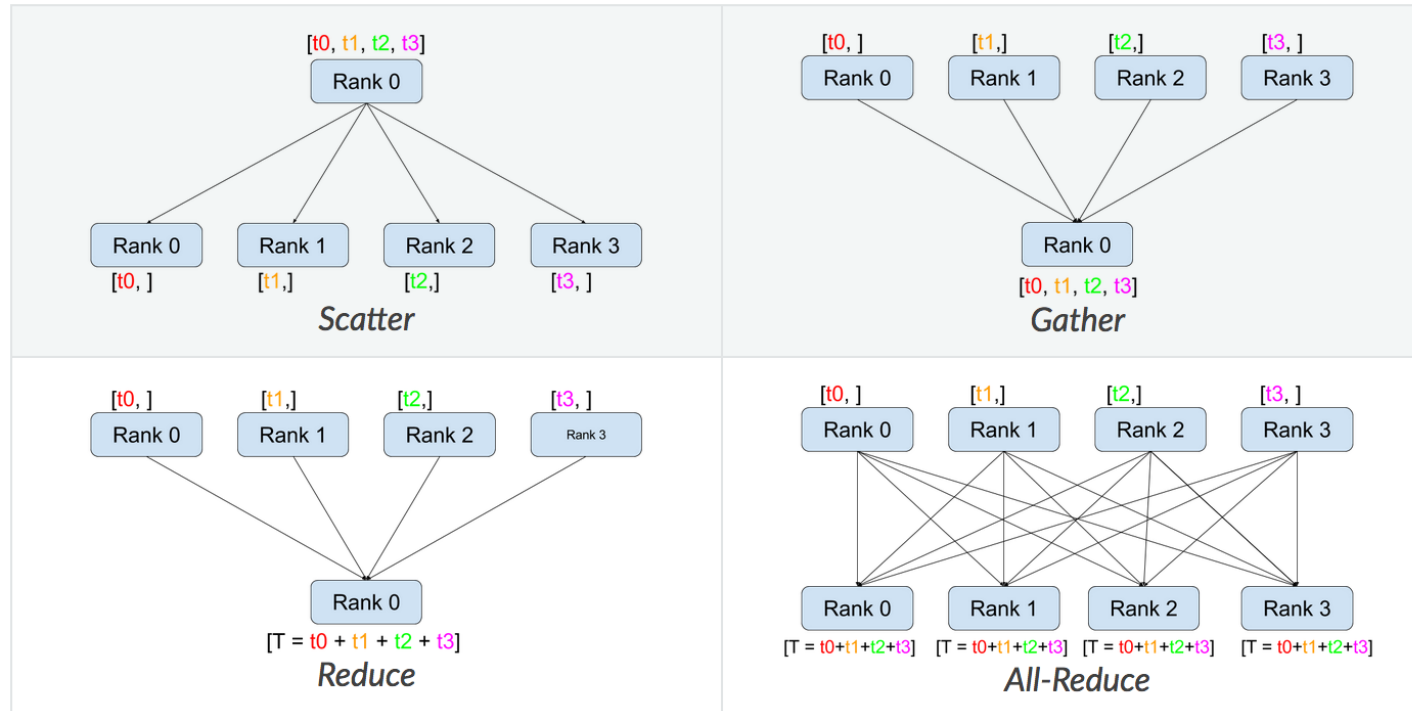
Optimizing communication efficiency is the key

Terminologies: Point-to-Point Primitive Communication

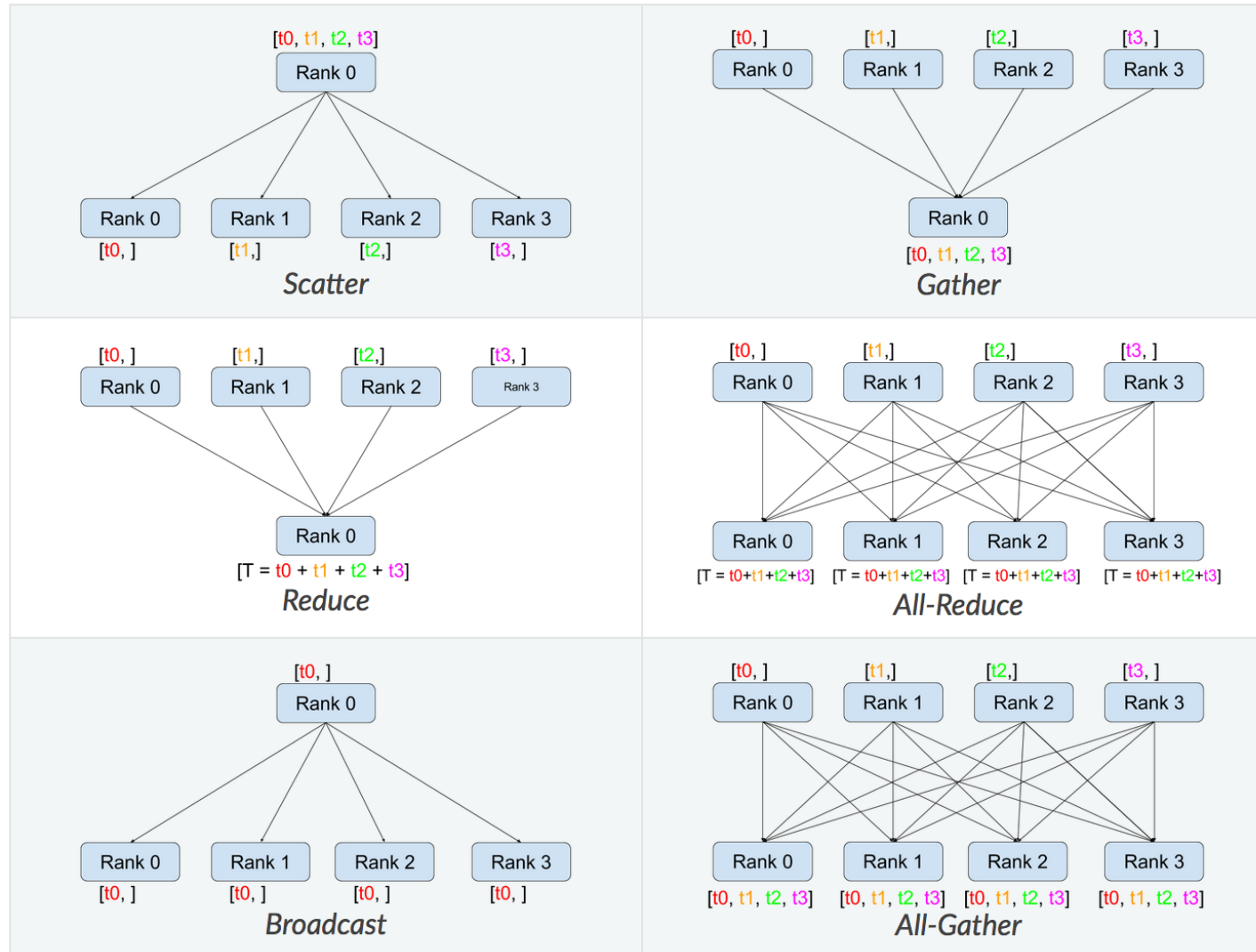




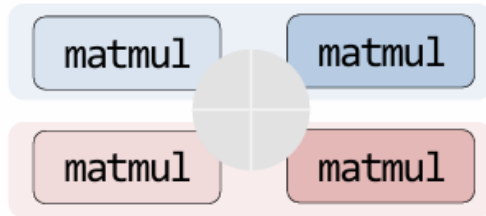
Terminologies: Collectives



Terminologies: Collectives



AllReduce is the Key in Data Parallelism

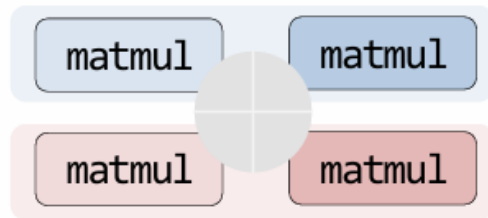


```
ddp_model = DDP(Model(), device_ids=[rank])  
for batch in data_loader:  
    loss = train_step(ddp_model, batch)
```



Implicit allreduce here

AllReduce is the Key in Data Parallelism



```
ddp_model = DDP(Model(), device_ids=[rank])  
for batch in data_loader:  
    loss = train_step(ddp_model, batch)
```



Implicit allreduce here

all-reduce

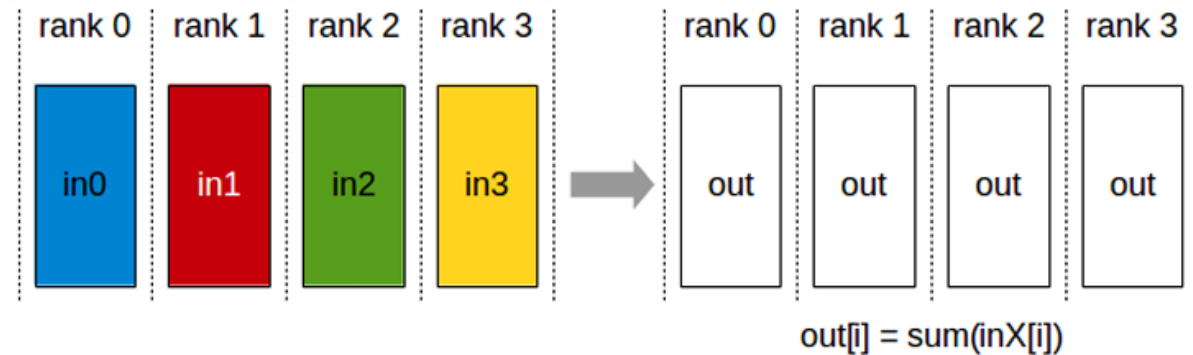
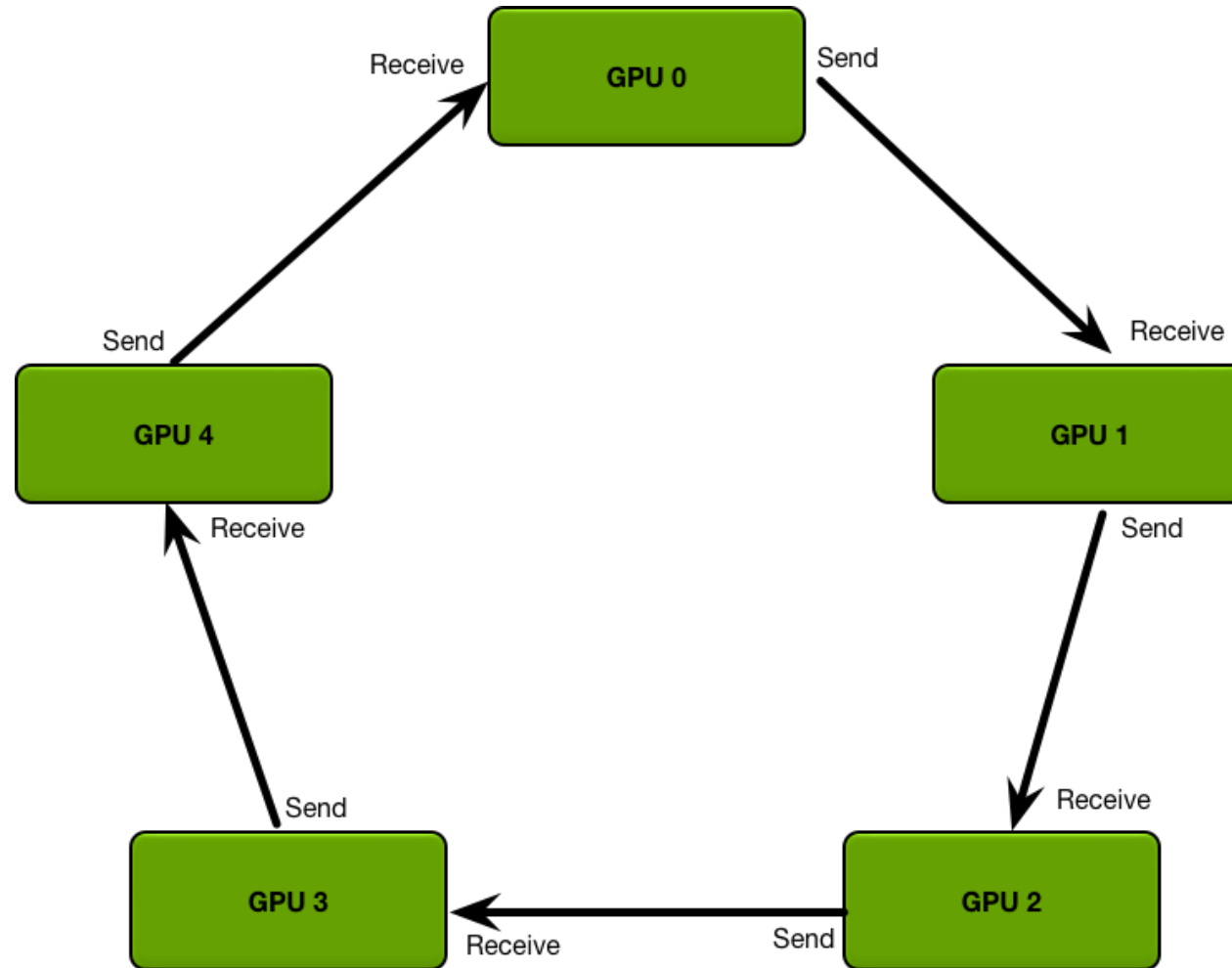


Figure from NCCL documentation

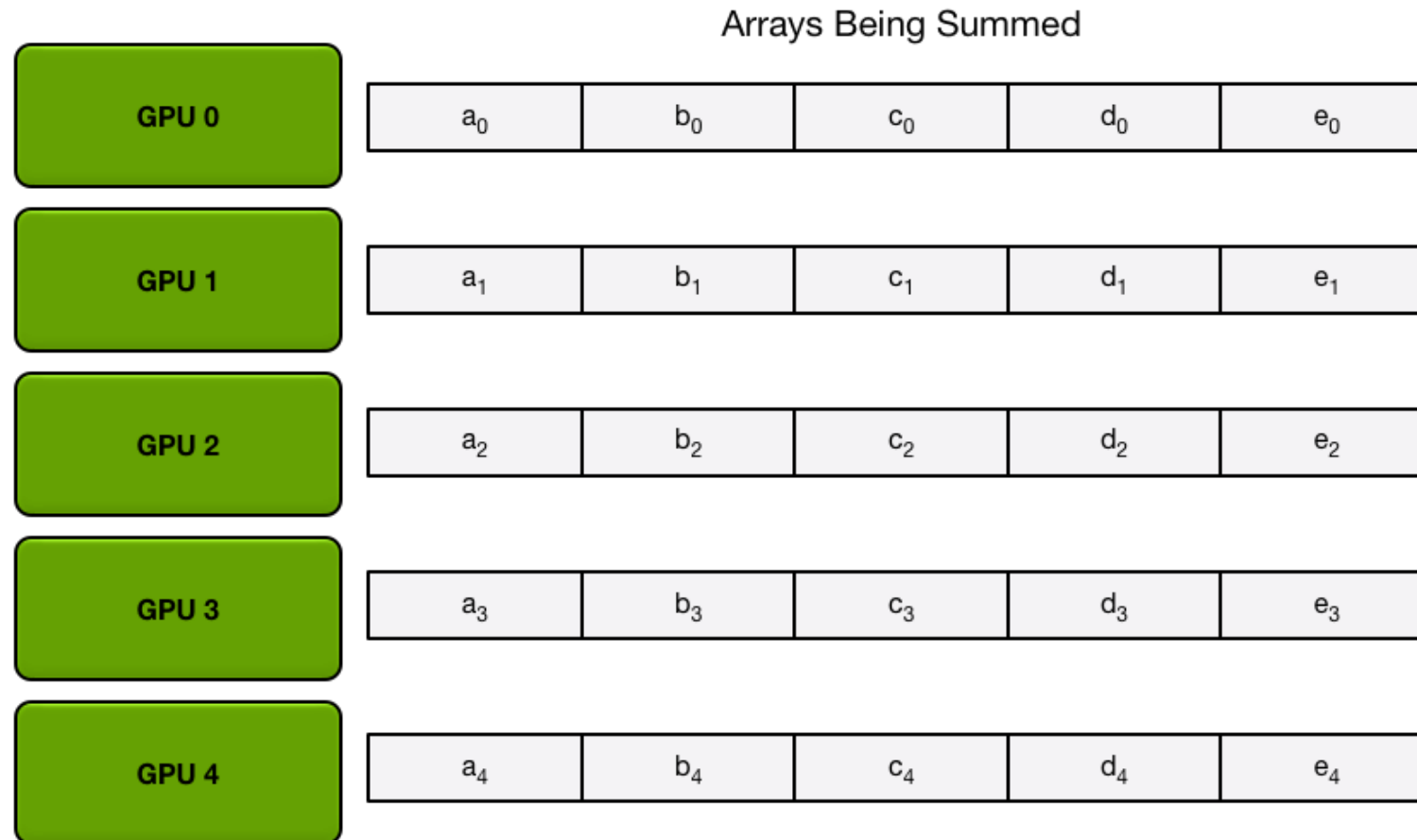
Ring-based AllReduce



GPUs arranged in a logical ring

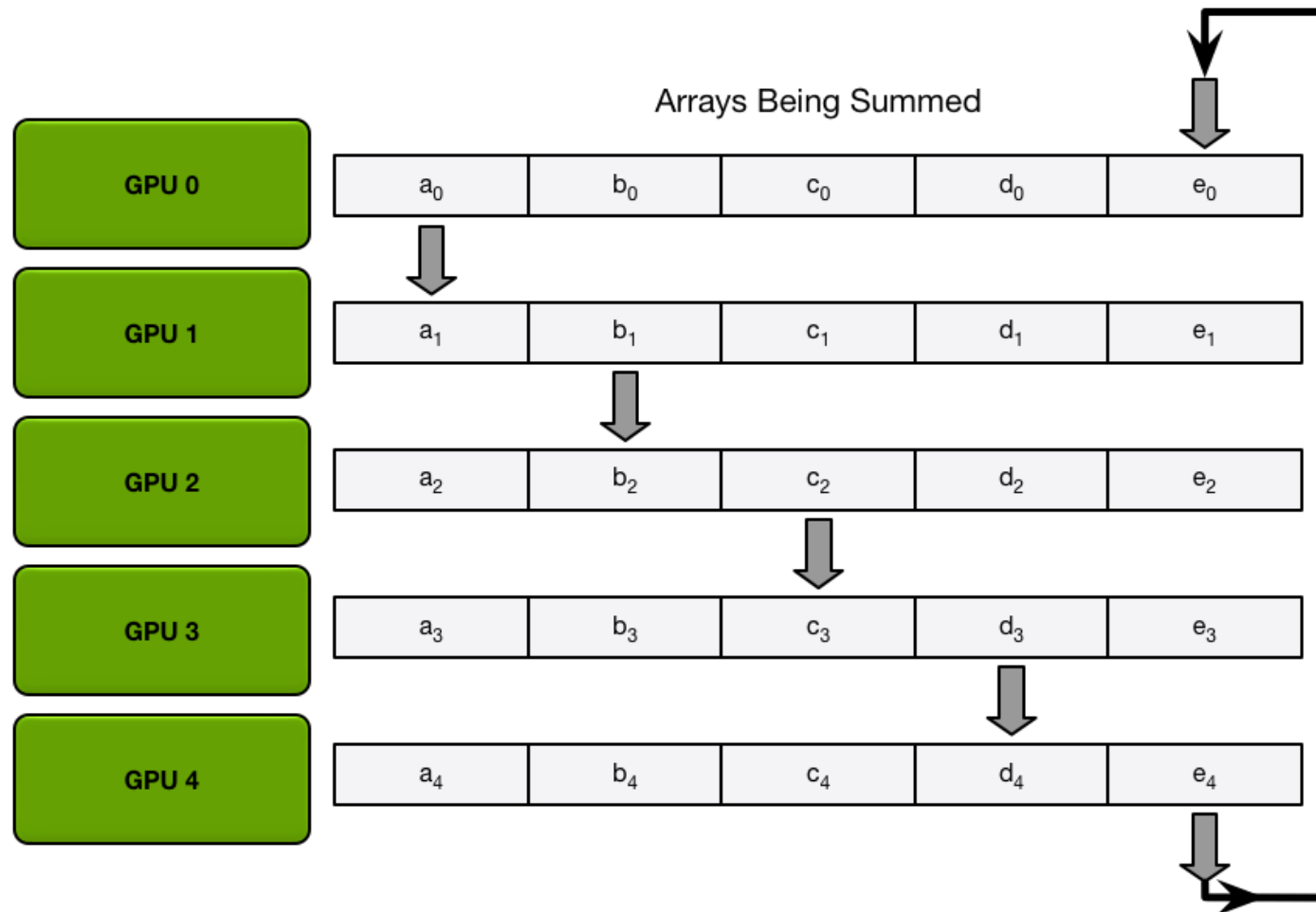
Two phases:
Scatter-reduce
All-gather

Phase 1: Reduce-Scatter



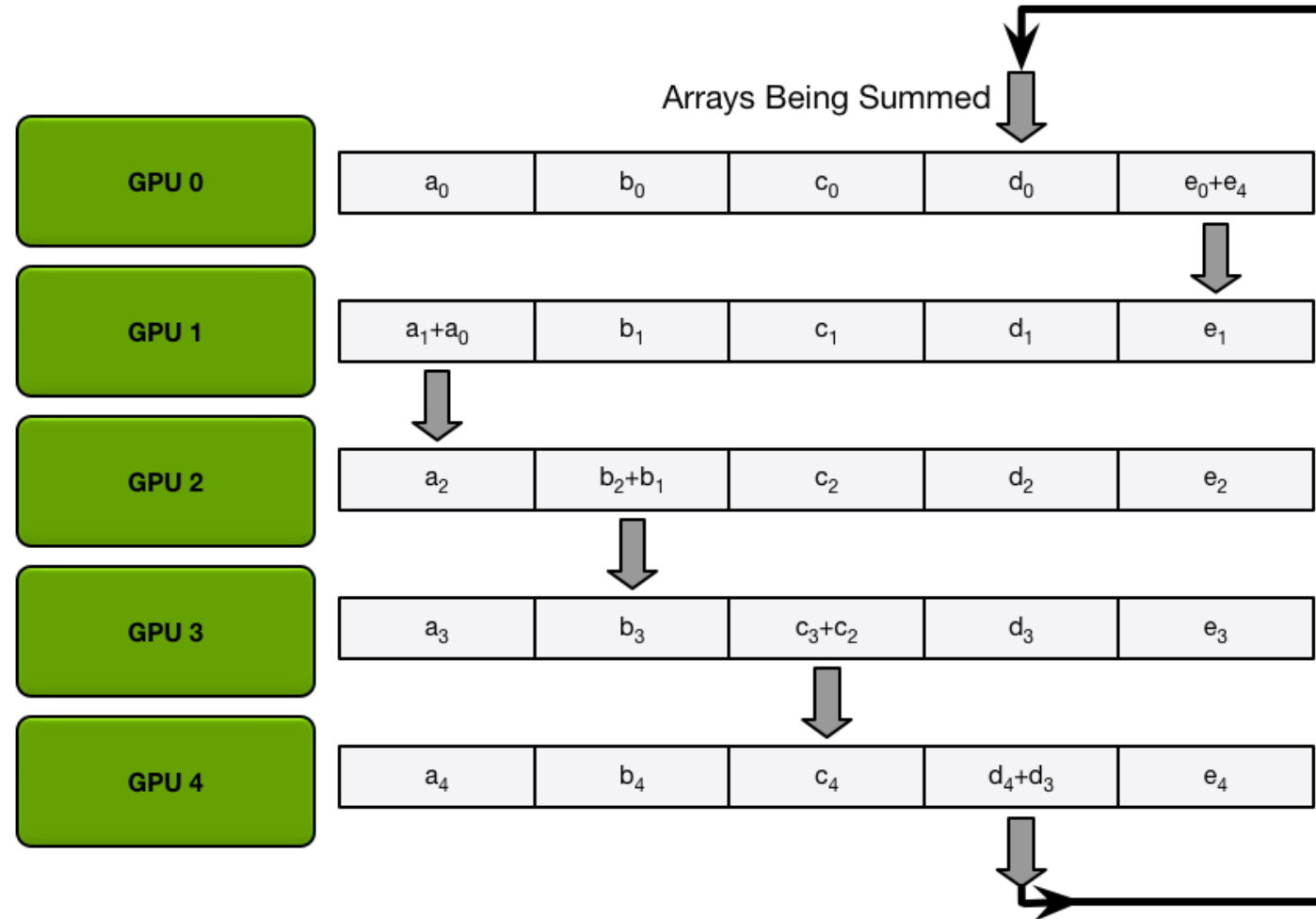
Partitioning of an array into N chunks

Phase 1: Reduce-Scatter



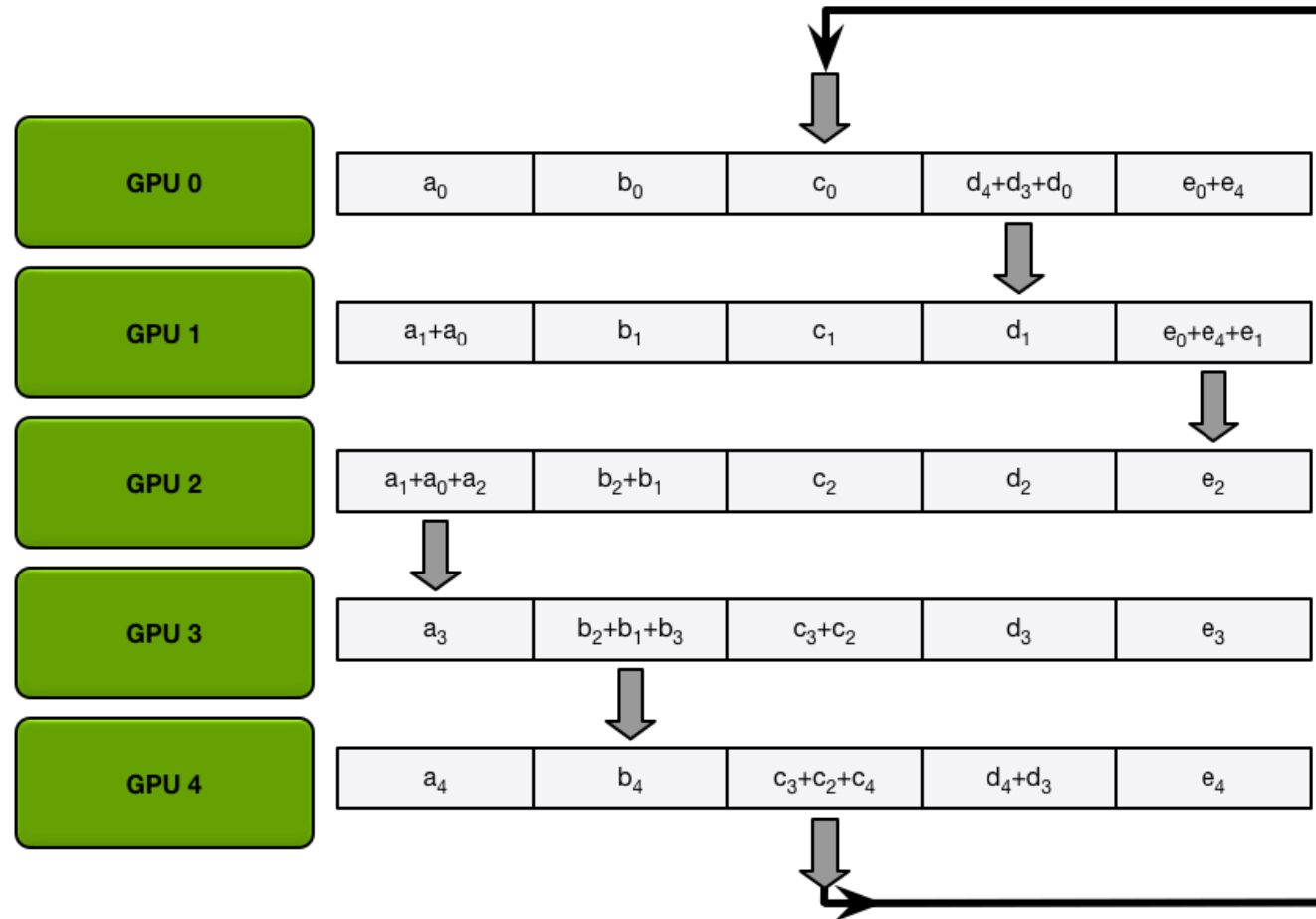
Data transfers in the 1st iteration of
scatter-reduce

Phase 1: Reduce-Scatter



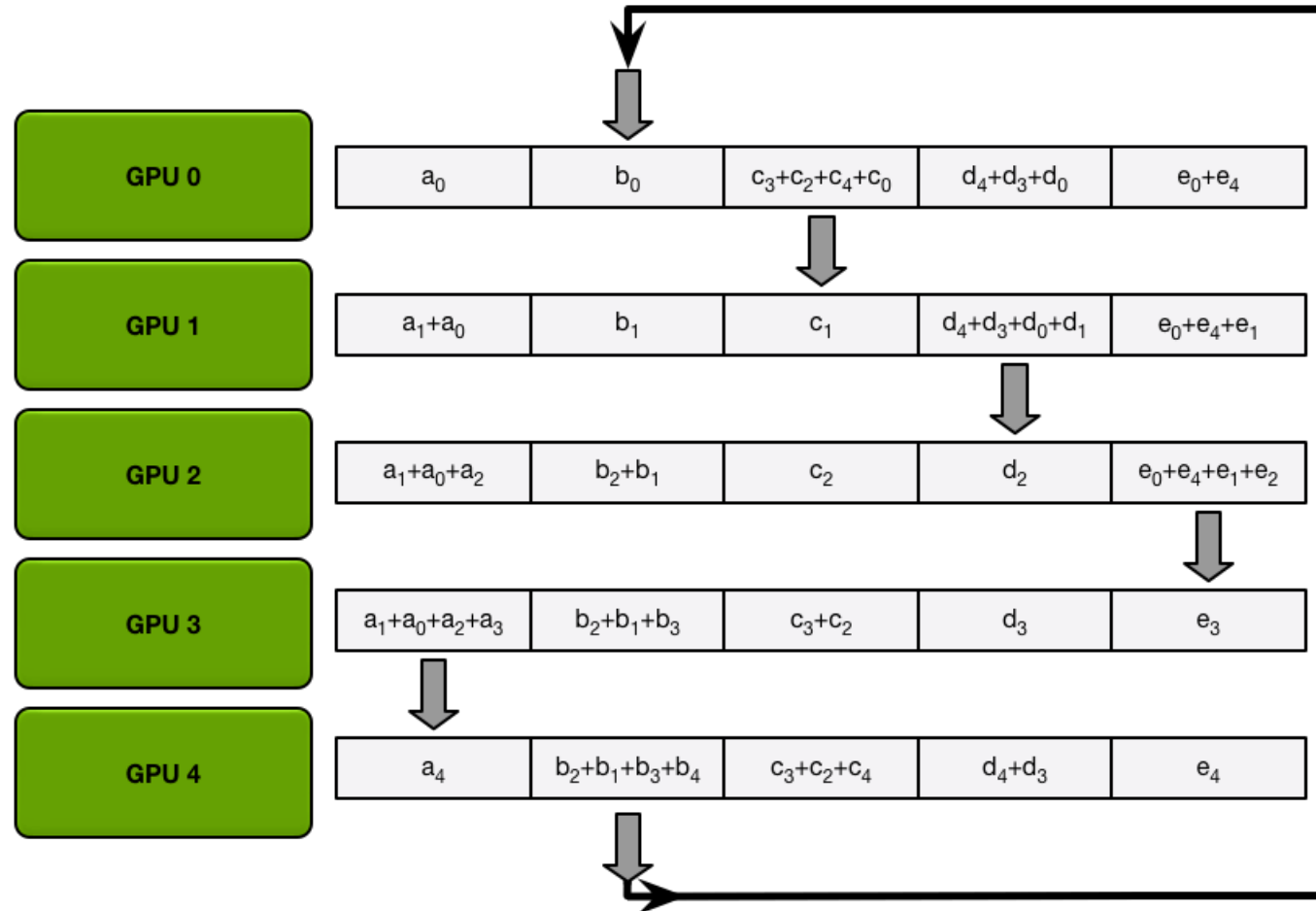
Intermediate sums after the first
iteration of scatter-reduce is complete

Phase 1: Reduce-Scatter



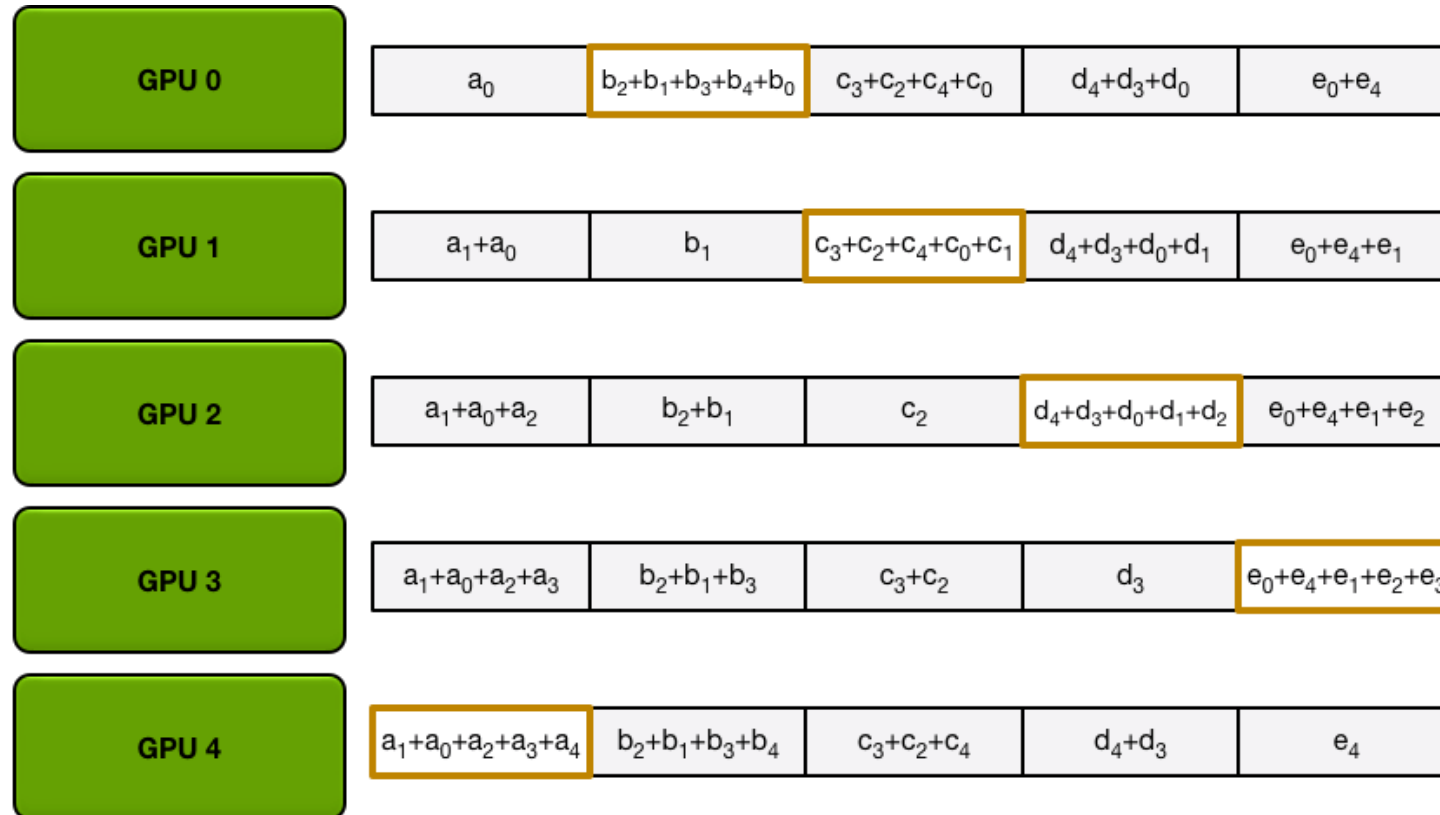
Reduce-scatter data transfer (iteration 3)

Phase 1: Reduce-Scatter



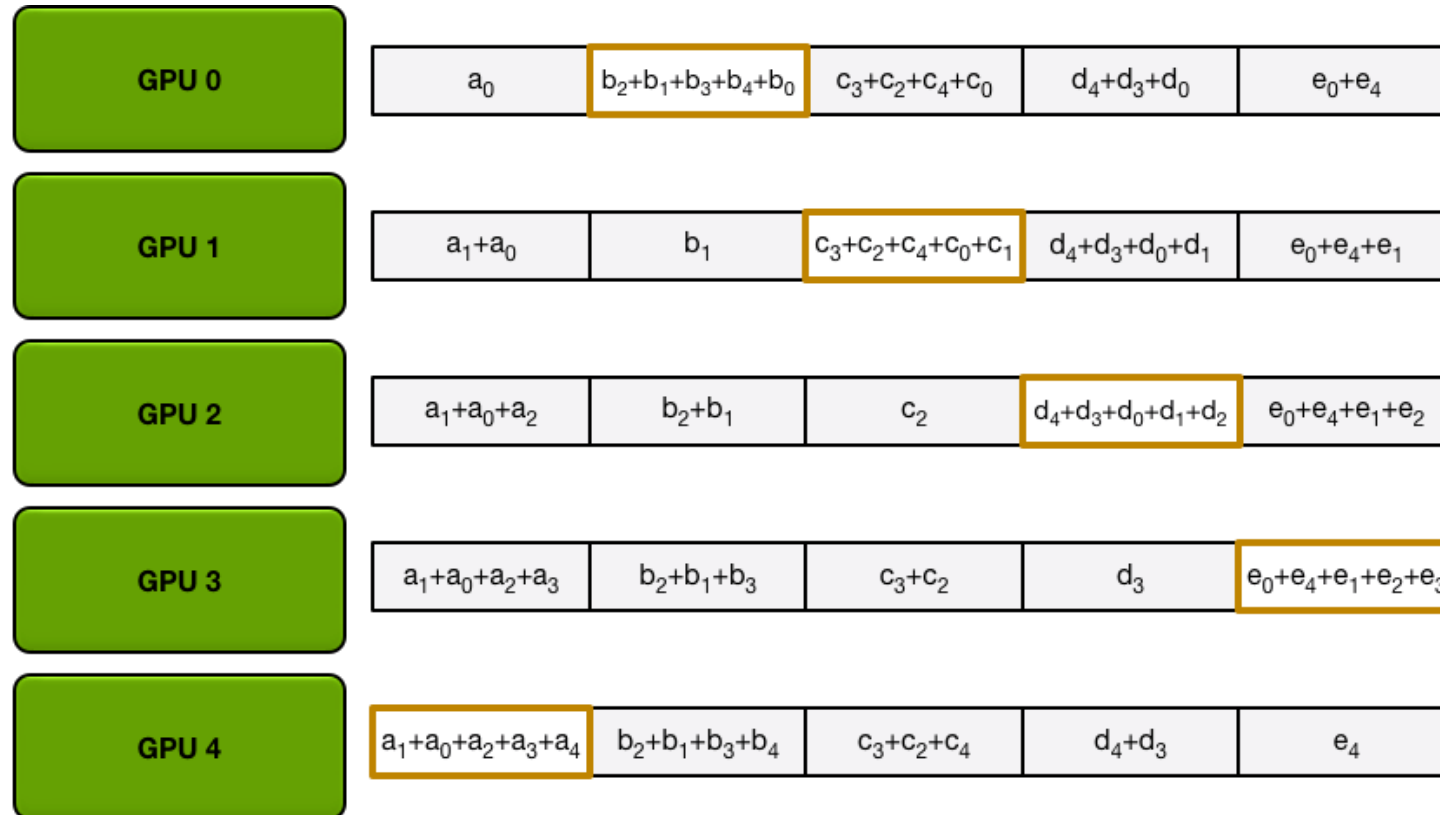
Reduce-scatter data transfer (iteration 4)

Phase 1: Reduce-Scatter



Reduce-scatter data transfer (after iteration 4)

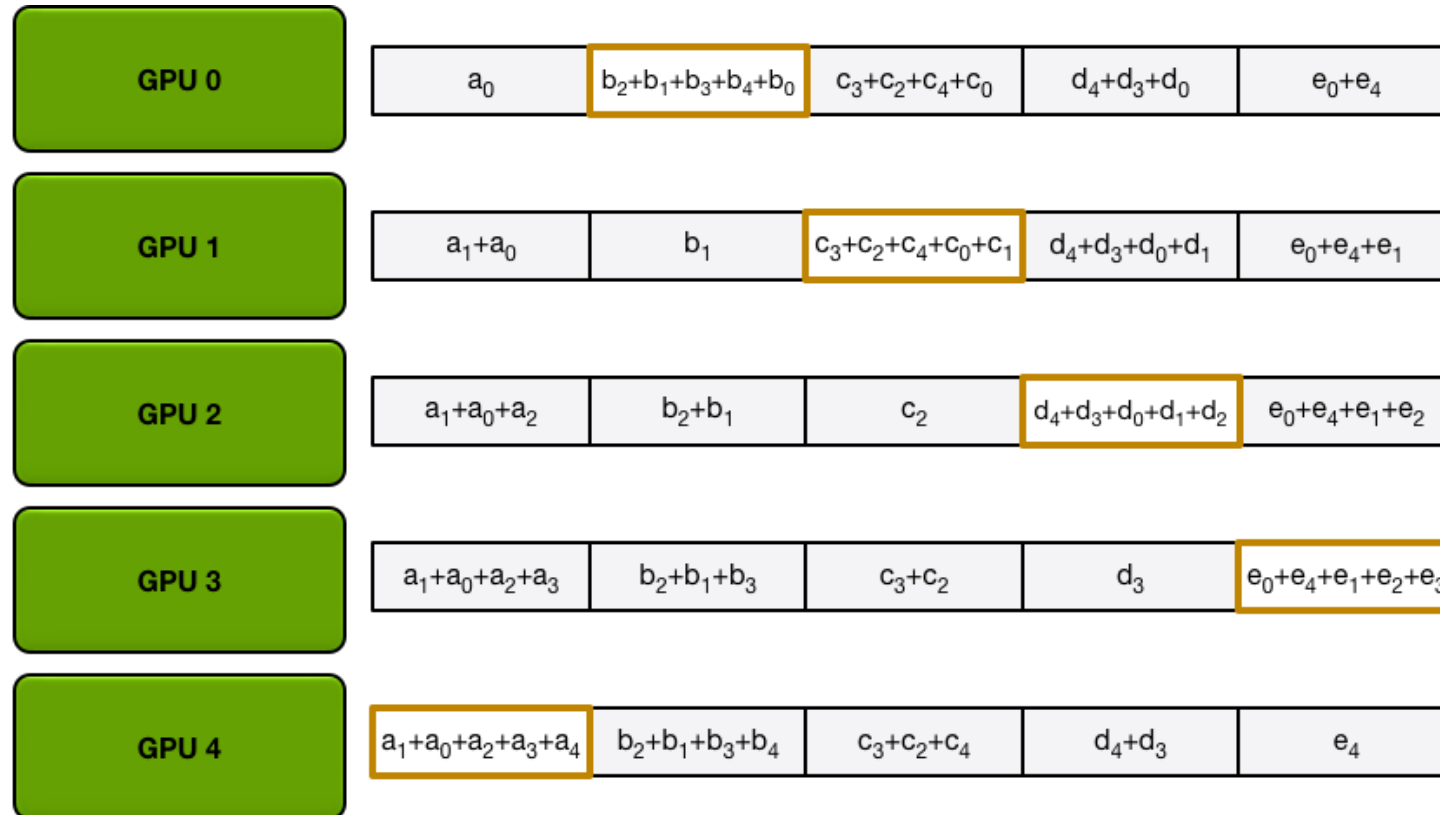
Phase 1: Reduce-Scatter



Question: What is the complexity of reduce-scatter?

Reduce-scatter data transfer (after iteration 4)

Phase 1: Reduce-Scatter



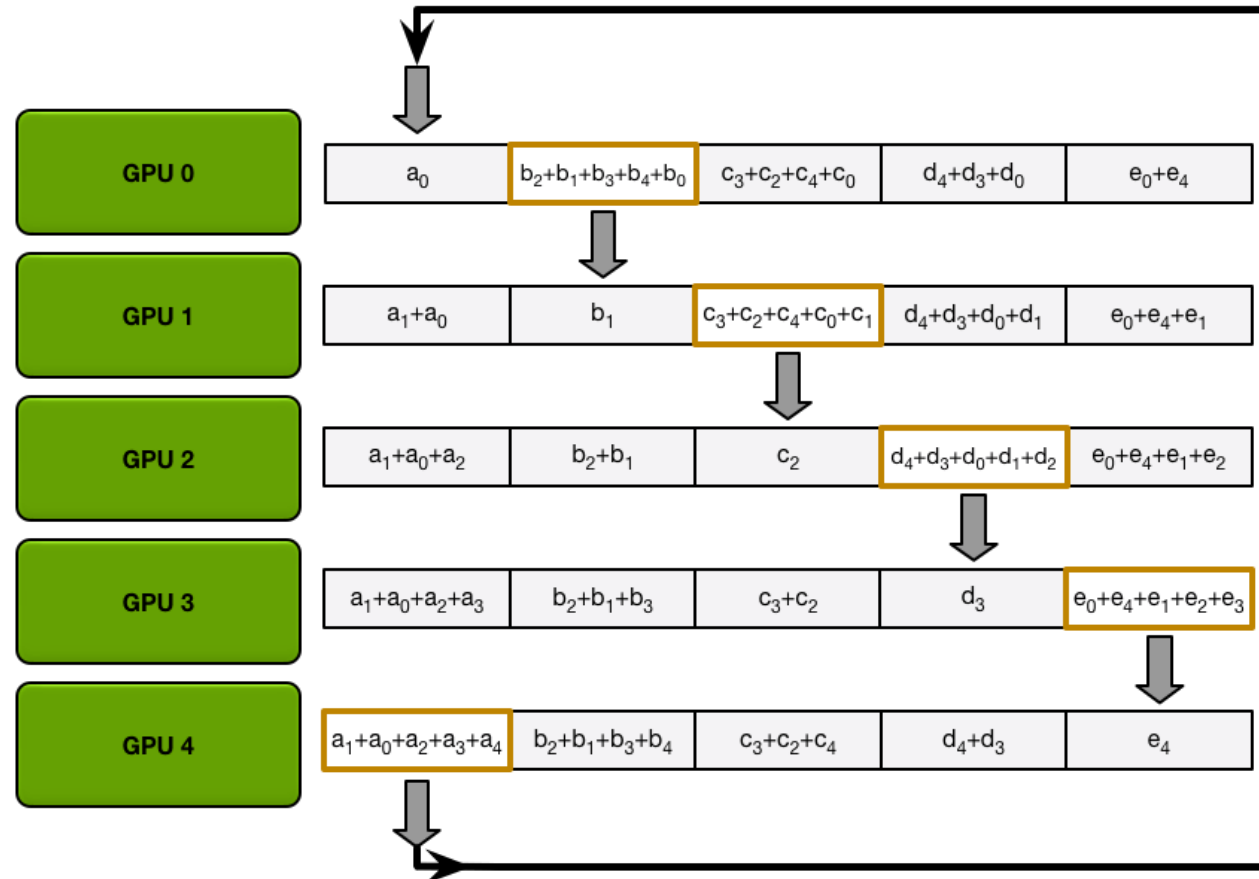
Question: What is the complexity of reduce-scatter?

- N: Size of data
- P: # of workers

$$O(N/P * (P - 1))$$

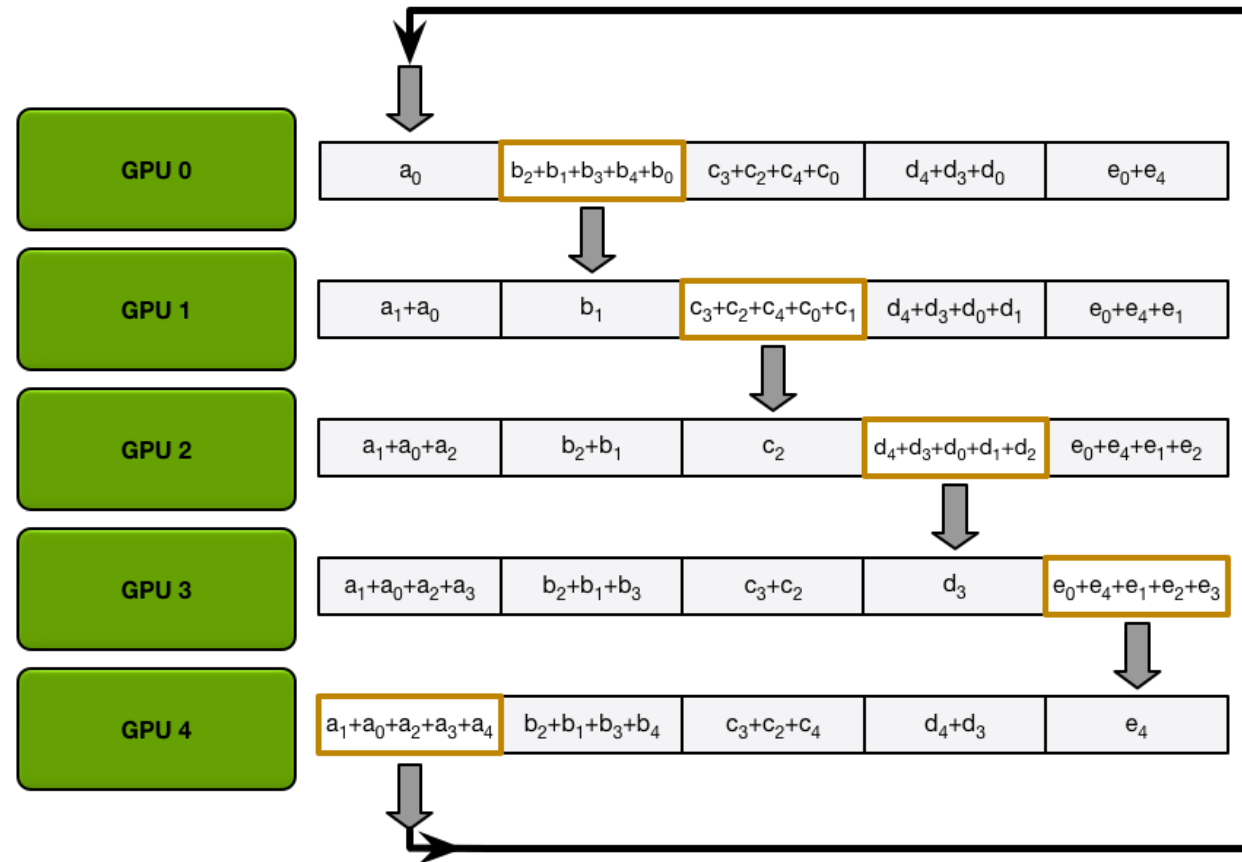
Reduce-scatter data transfer (after iteration 4)

Phase 2: Allgather



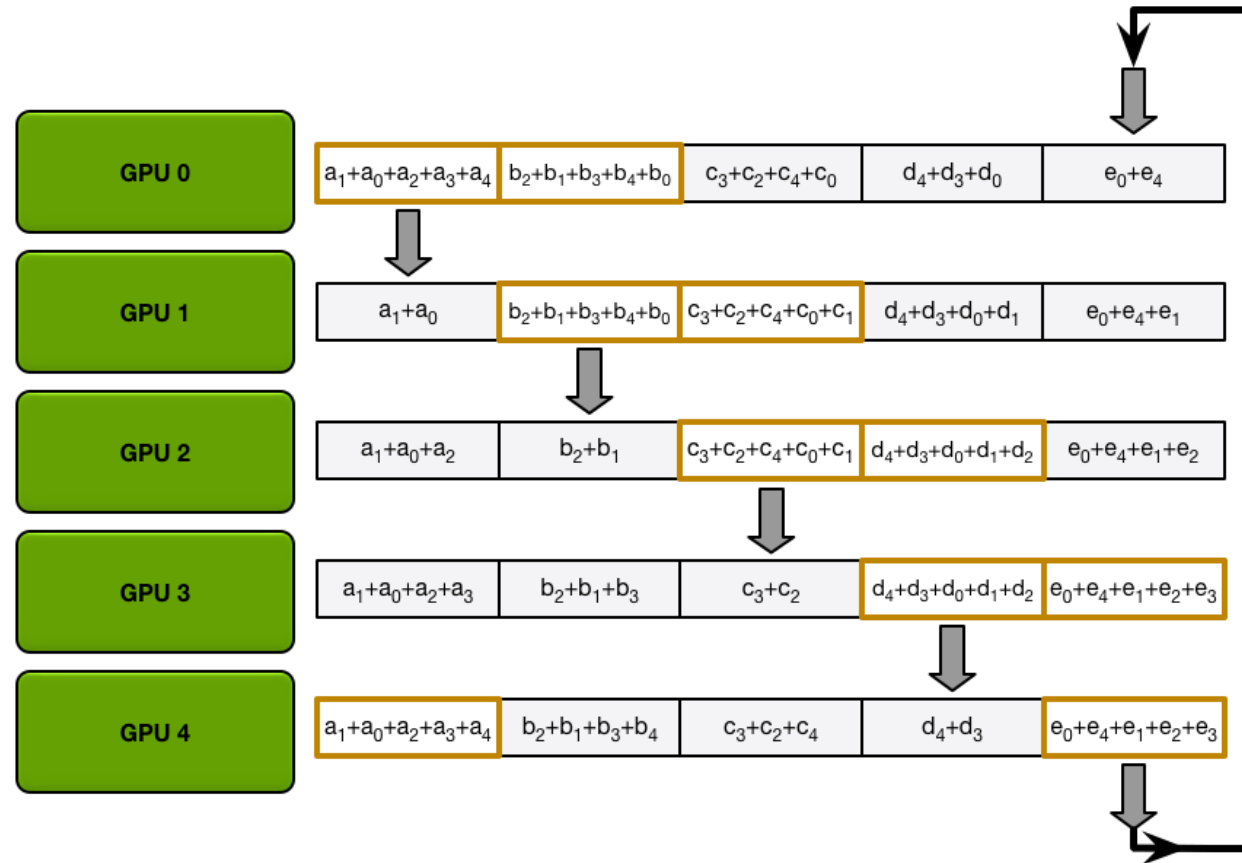
All processes can obtain the complete array by circulating again without reduction operations.

Phase 2: Allgather



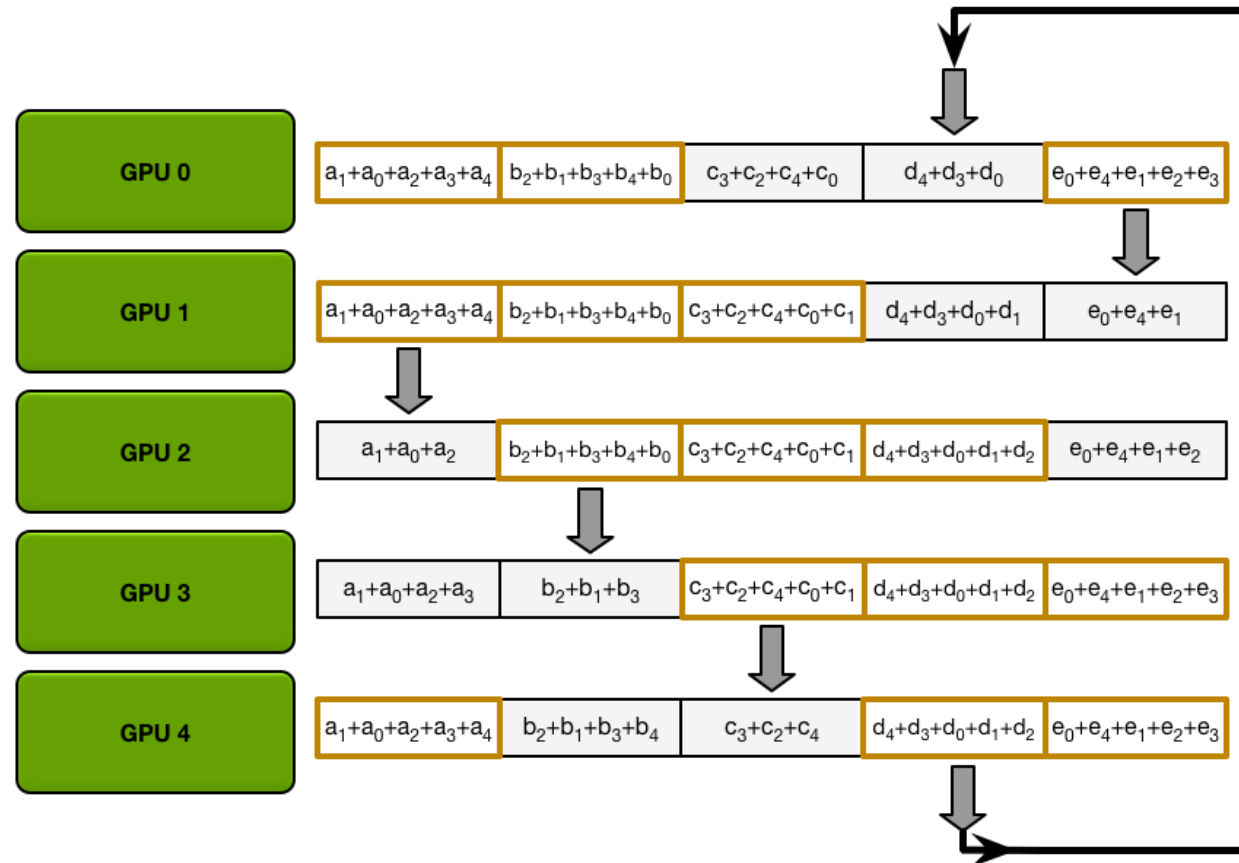
Allgather data transfer (iteration 1)

Phase 2: Allgather



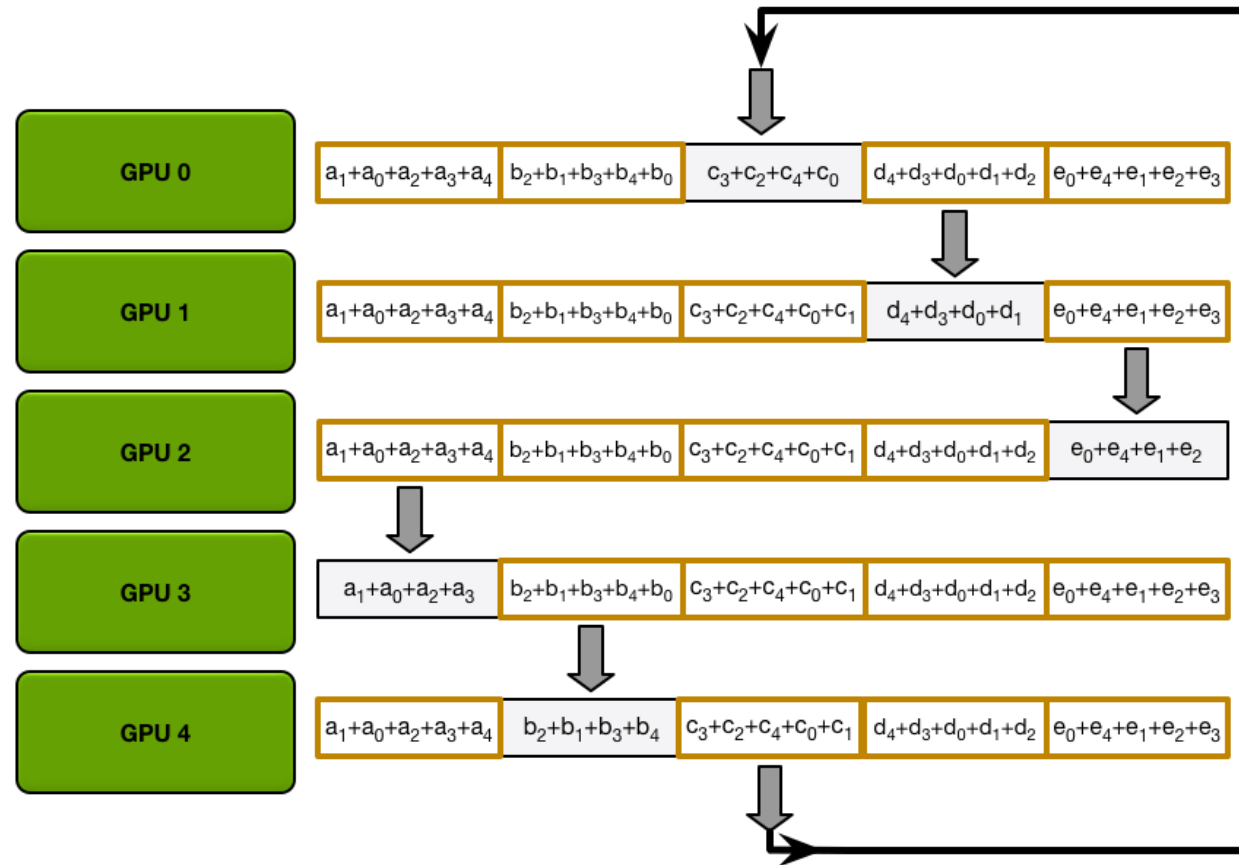
Allgather data transfer (iteration 2)

Phase 2: Allgather



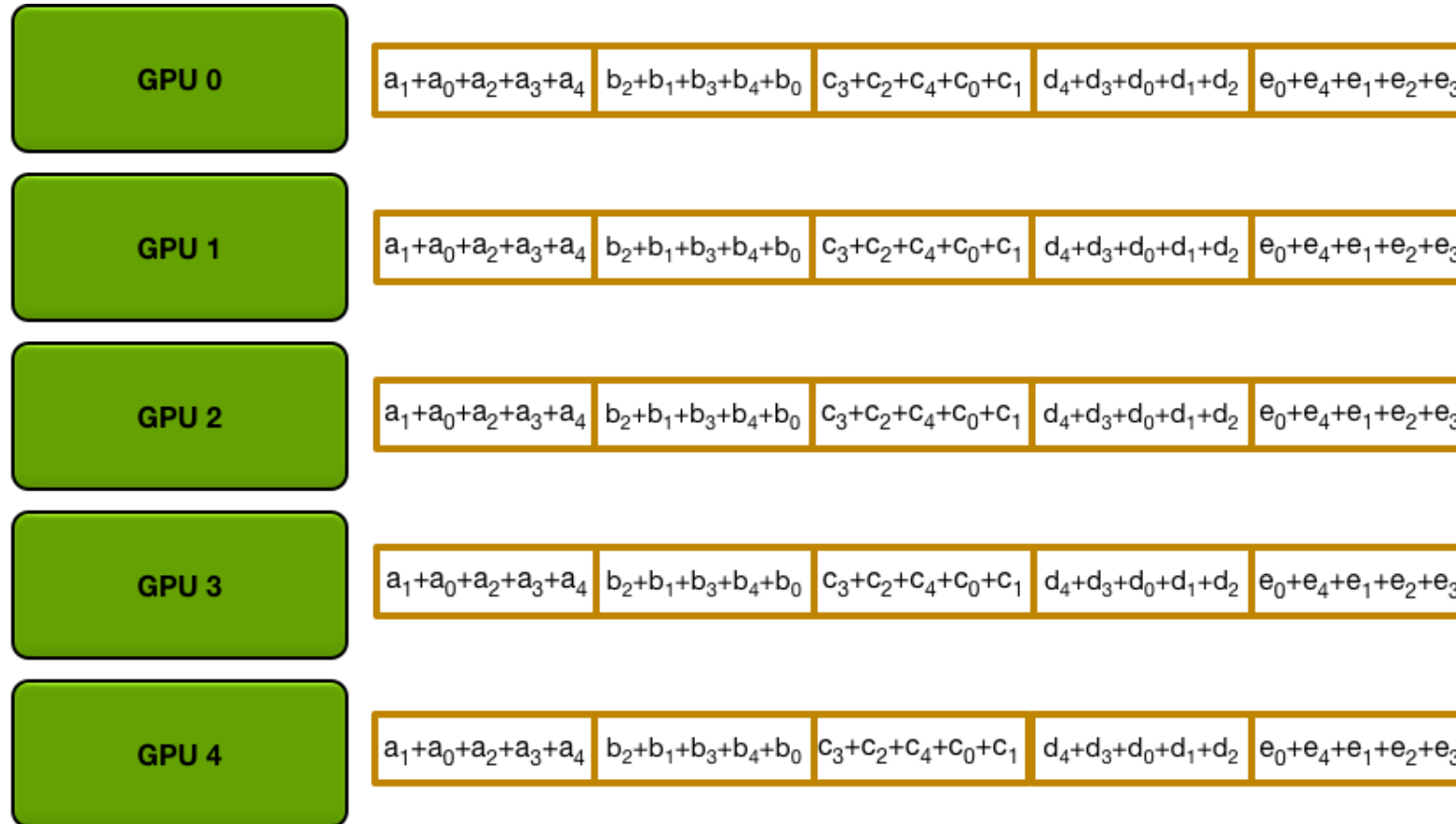
Allgather data transfer (iteration 3)

Phase 2: Allgather



Allgather data transfer (iteration 4)

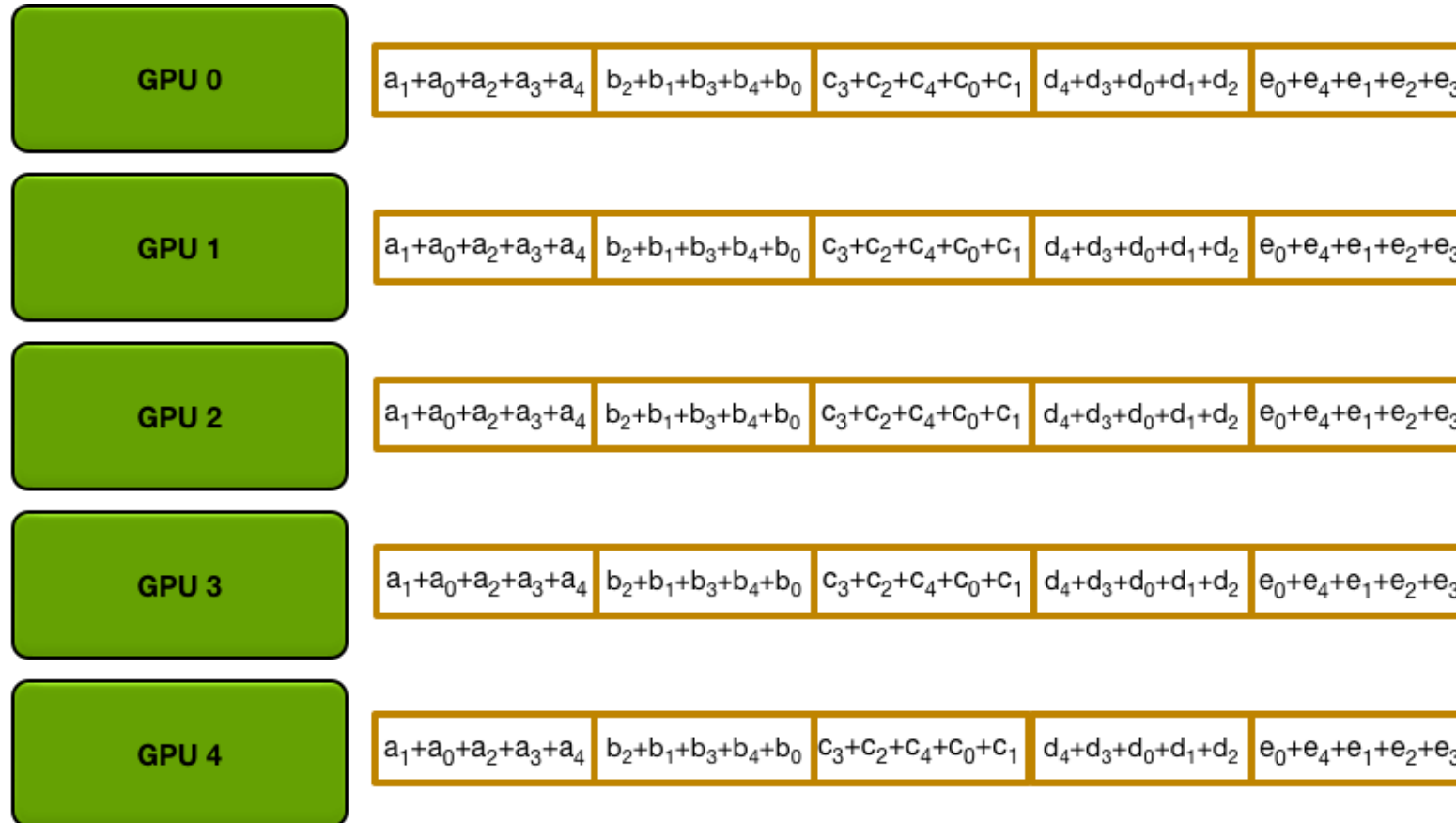
Phase 2: Allgather



Question: What is the time complexity of all-gather?

Final state after allgather transfer

Phase 2: Allgather



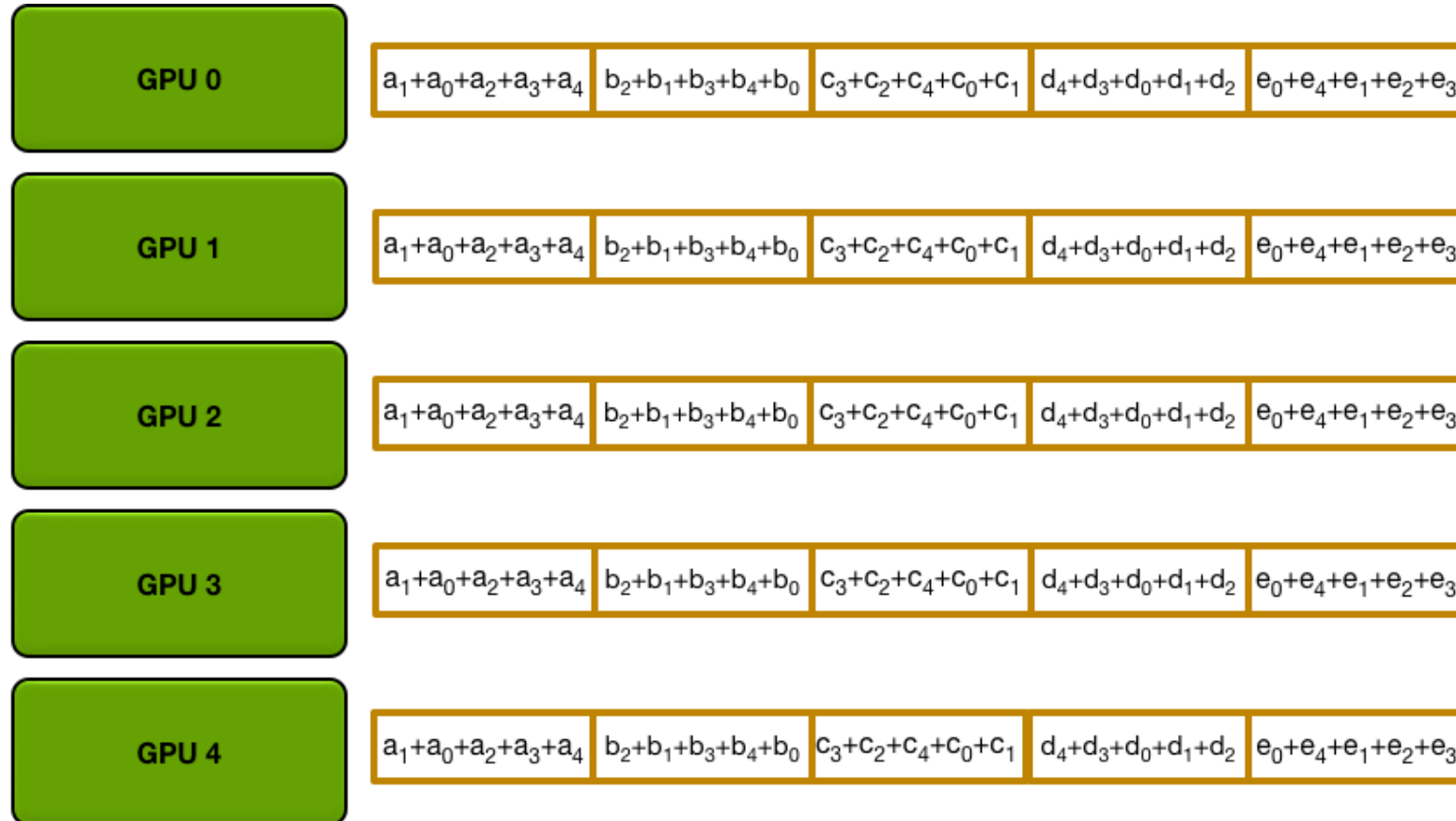
Question: What is the time complexity of all-gather?

- N: Size of data
- P: # of workers/processes

$$O(N/P * (P - 1))$$

Final state after allgather transfer

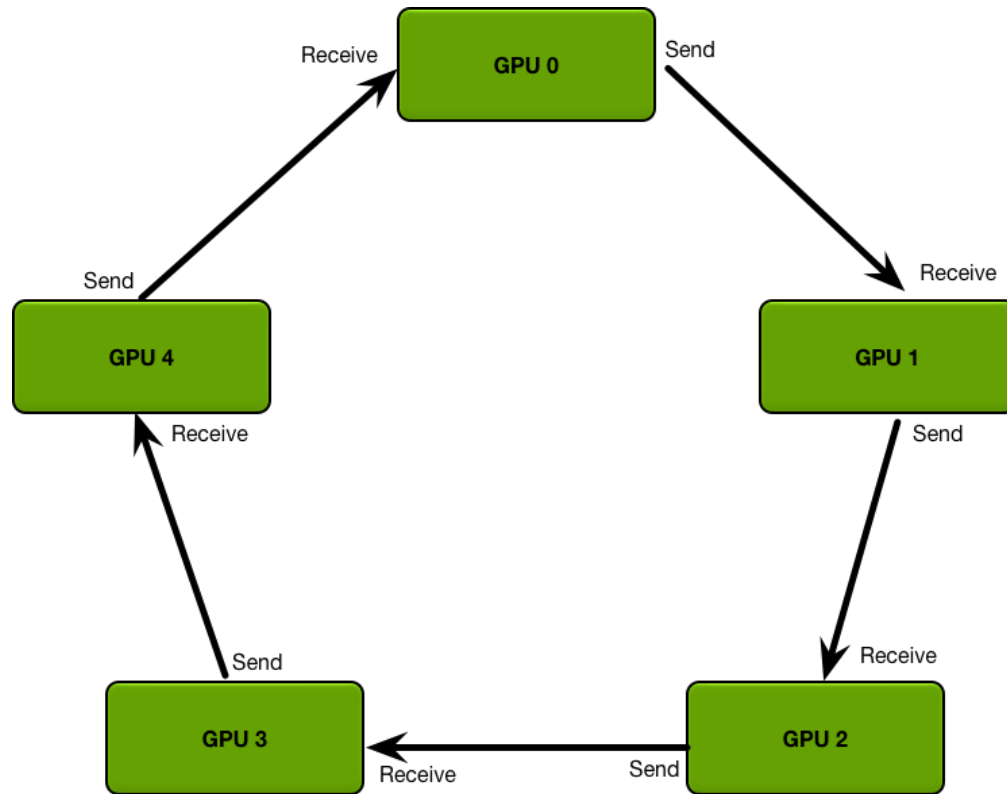
Phase 2: Allgather



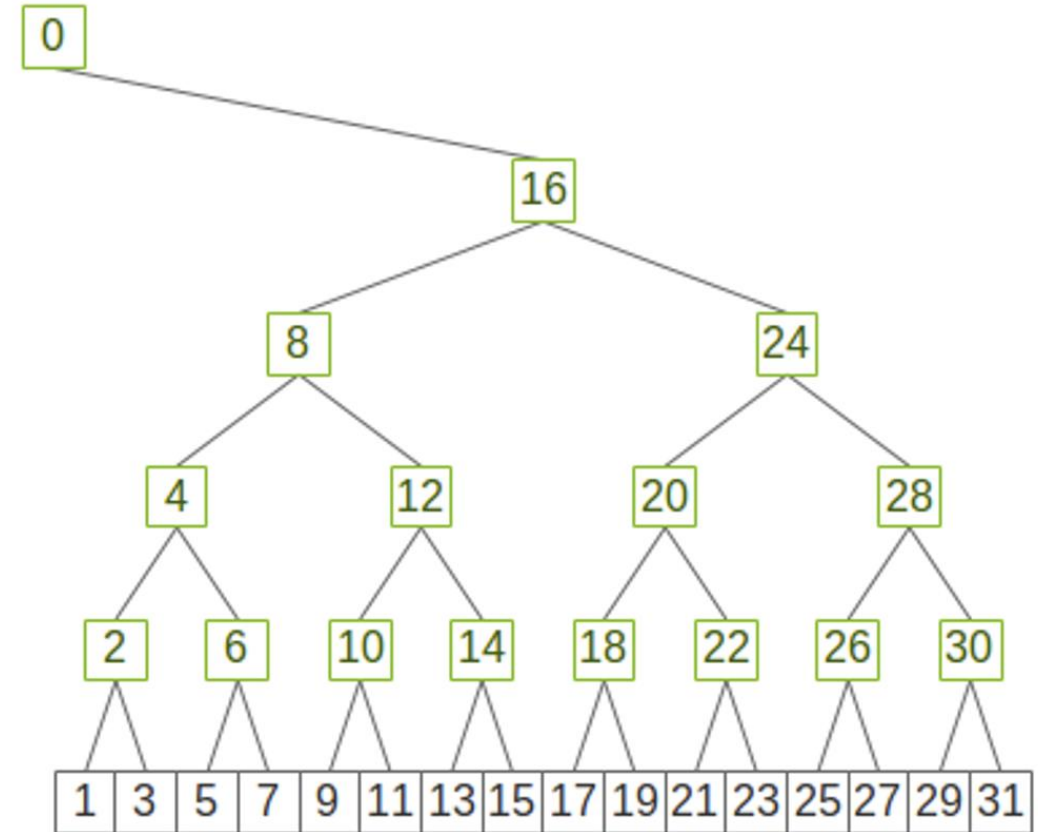
First phase: $N/P * (P - 1)$
Second phase: $N/P * (P - 1)$
Total: $2N (P - 1) / P$, largely independent of P

Final state after allgather transfer

Two AllReduce Paradigms

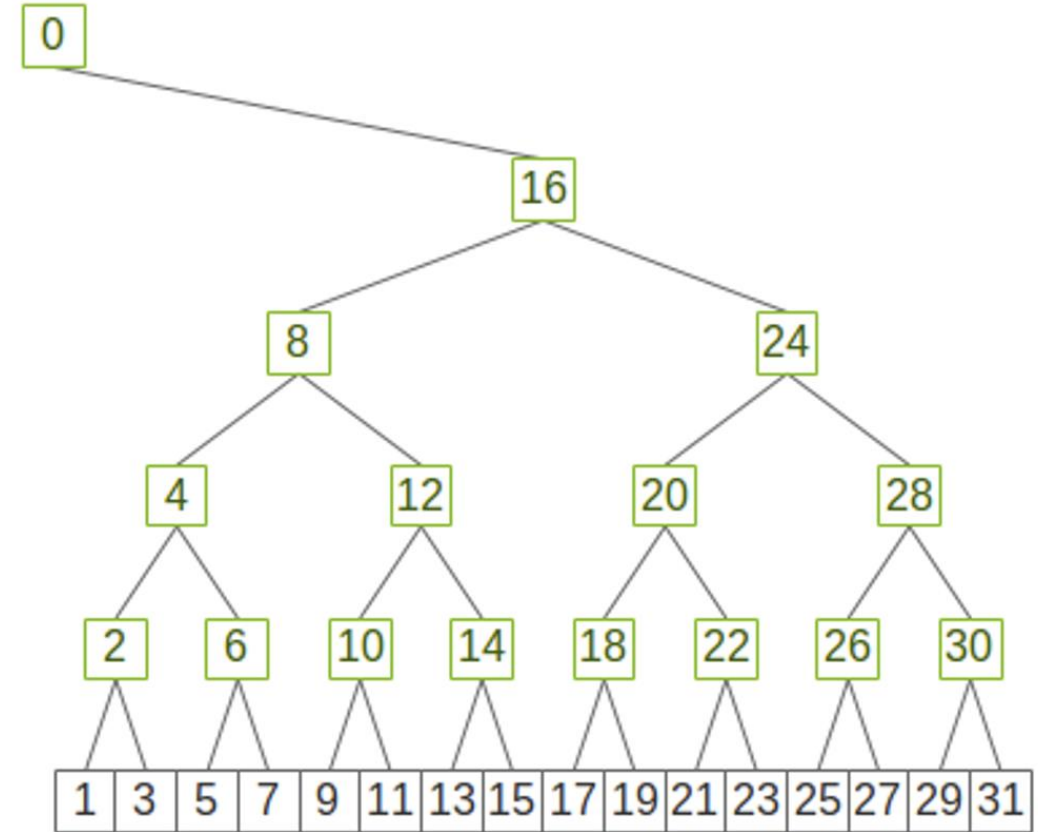
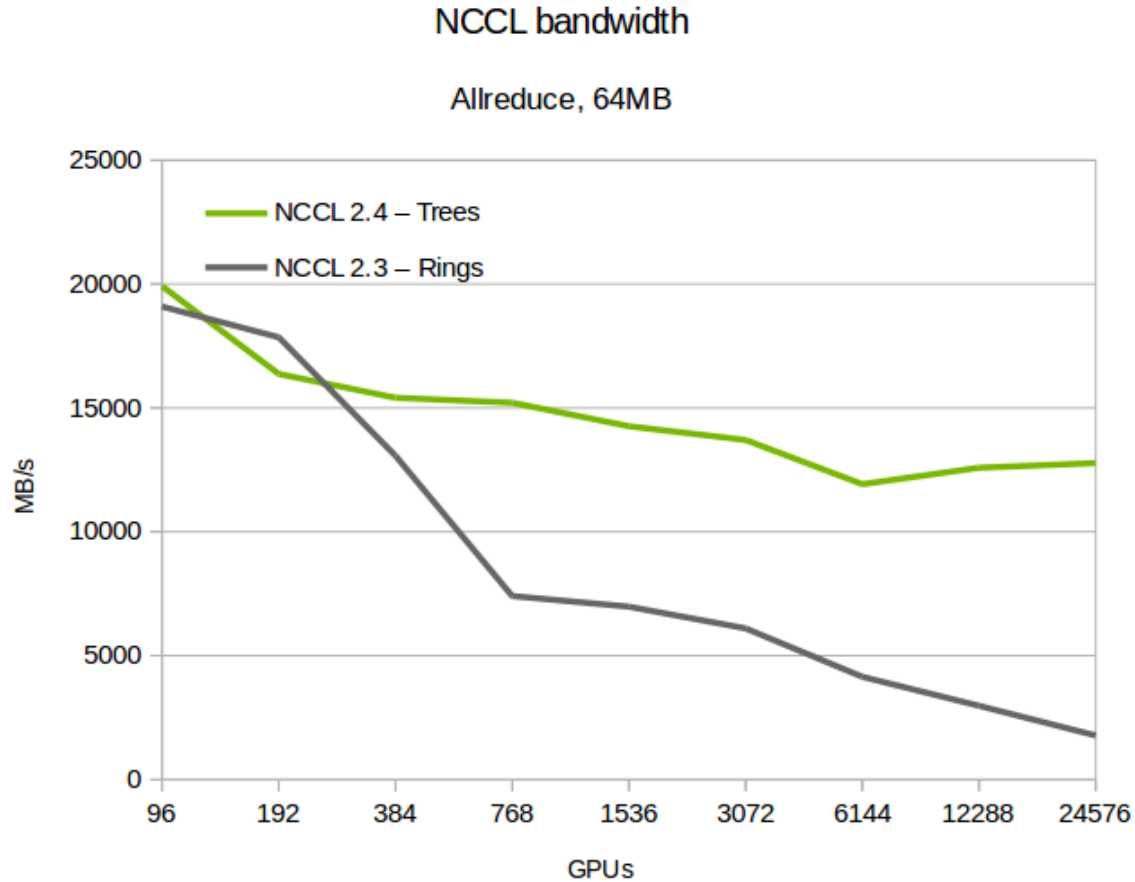


Ring-based



Tree-based

Two AllReduce Paradigms



Tree-based

Reduce-scatter

$$(p-1)\alpha + \frac{p-1}{p}n(\beta + \gamma)$$

Scatter

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Gather

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Allgather

$$(p-1)\alpha + \frac{p-1}{p}n\beta$$

α : Round-trip time latency

β : Network bandwidth

γ : Compute overhead

Reduce(-to-one)

Allreduce

Broadcast

α : Round-trip time latency
 β : Network bandwidth
 γ : Compute overhead

Reduce-scatter

$$(p-1)\alpha + \frac{p-1}{p}n(\beta + \gamma)$$

Scatter

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Gather

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Allgather

$$(p-1)\alpha + \frac{p-1}{p}n\beta$$

Reduce(-to-one)

$$(p-1+\log(p))\alpha + \frac{p-1}{p}n(2\beta + \gamma)$$

Allreduce

$$2(p-1)\alpha + \frac{p-1}{p}n(2\beta + \gamma)$$

Broadcast

α : Round-trip time latency
 β : Network bandwidth
 γ : Compute overhead

Reduce-scatter

$$(p-1)\alpha + \frac{p-1}{p}n(\beta + \gamma)$$

Scatter

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Gather

$$\log(p)\alpha + \frac{p-1}{p}n\beta$$

Allgather

$$(p-1)\alpha + \frac{p-1}{p}n\beta$$

Reduce(-to-one)

$$(p-1 + \log(p))\alpha + \frac{p-1}{p}n(2\beta + \gamma)$$

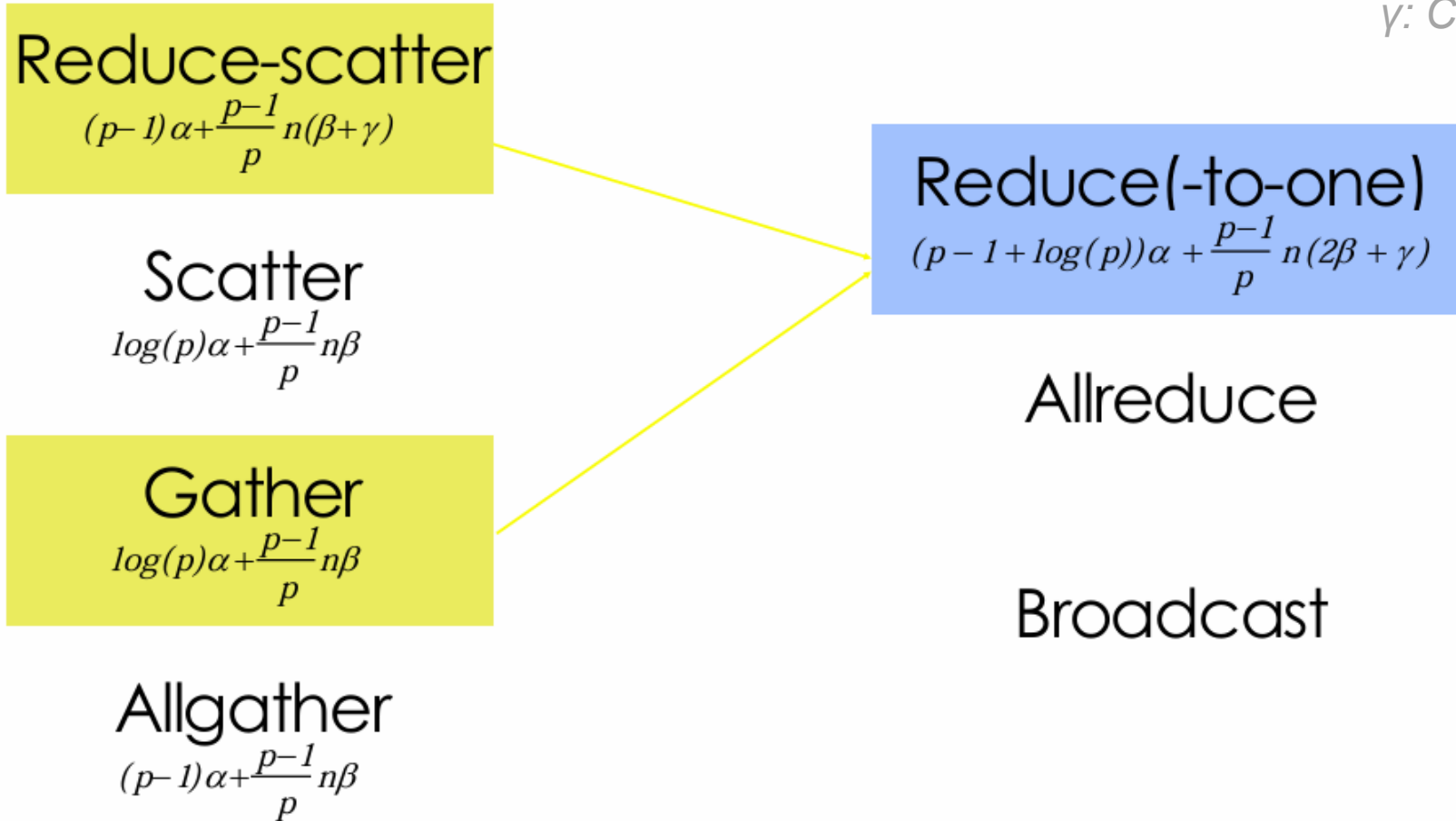
Allreduce

$$2(p-1)\alpha + \frac{p-1}{p}n(2\beta + \gamma)$$

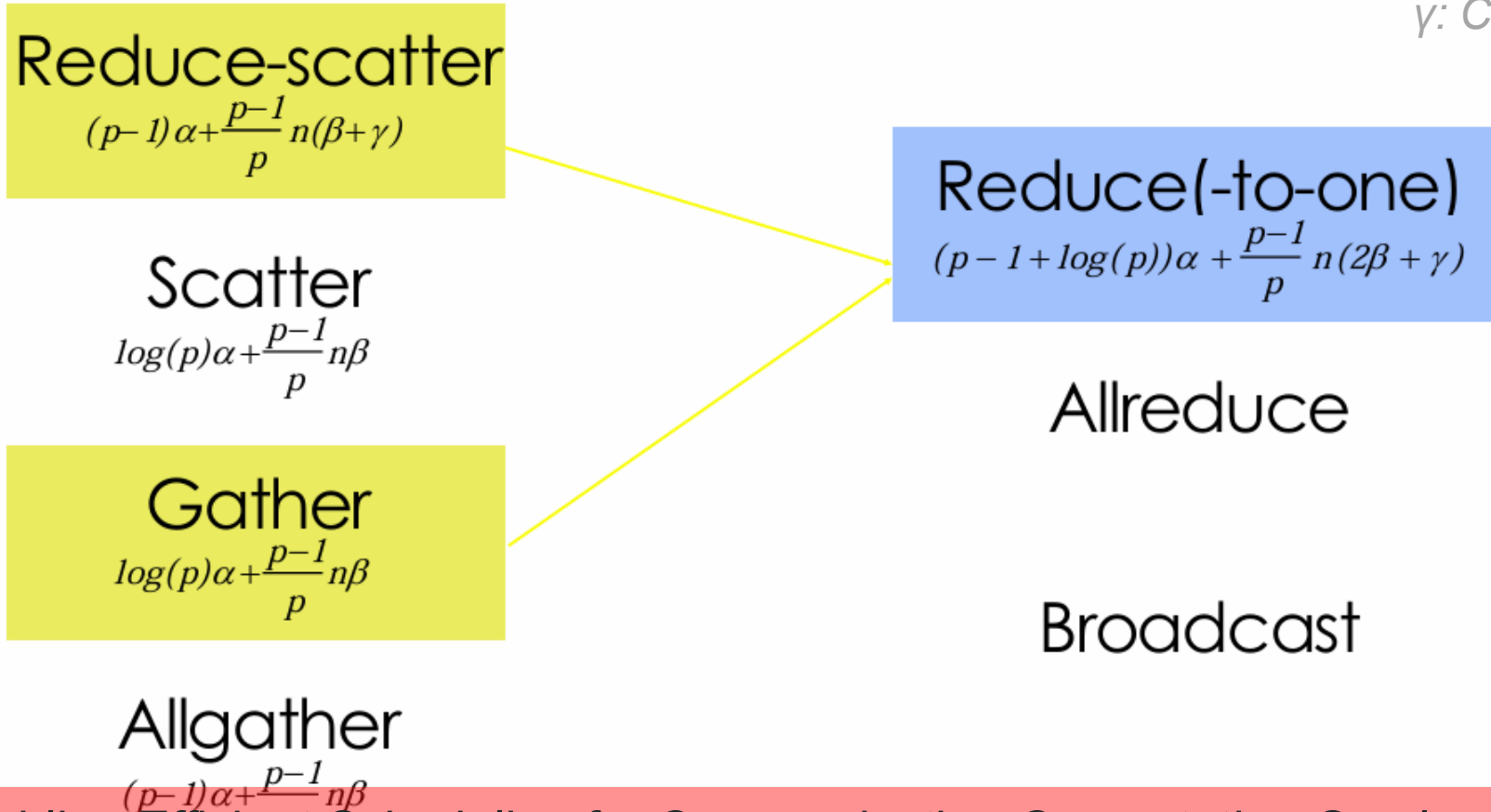
Broadcast

$$(\log(p) + p-1)\alpha + 2\frac{p-1}{p}n\beta$$

α : Round-trip time latency
 β : Network bandwidth
 γ : Compute overhead



α : Round-trip time latency
 β : Network bandwidth
 γ : Compute overhead



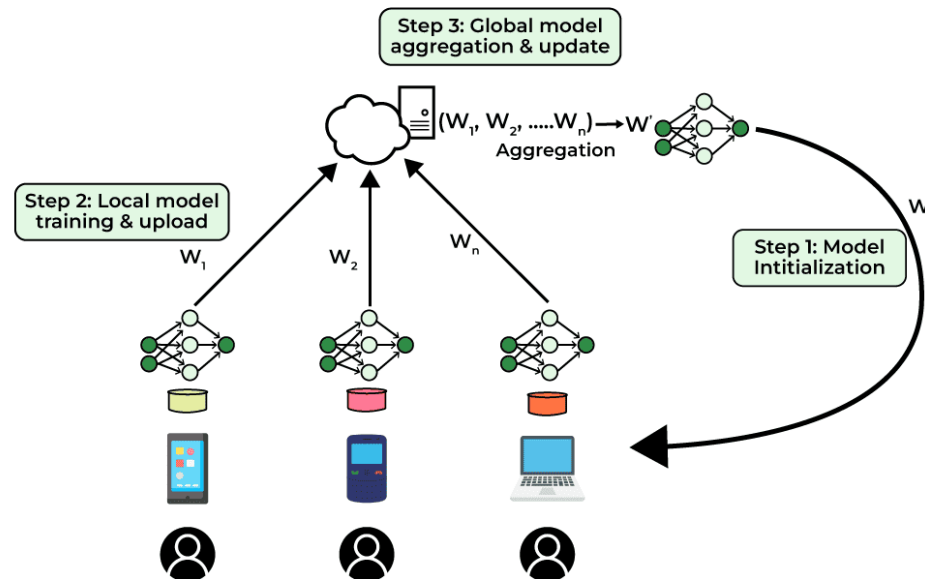
“Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning” ---ASPLOS’24 (Best Paper)

- Collectives are implemented using many P2P
 - Collectives are much more expensive than P2P
- Collectives are highly optimized throughout the past 20 years
 - Look for “X”CCL libraries
 - NCCL, RCCL, MSCCL,...
- Collectives are not fault-tolerant
- Collectives can be implemented differently
 - Ring
 - Minimum Spanning Tree (MST)

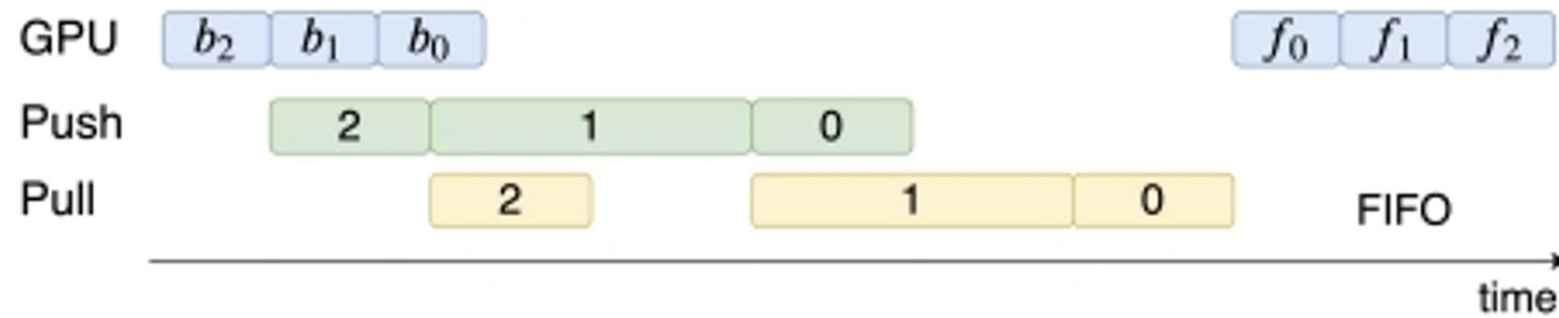
- Use unified GPU direct memory accessing
- Each GPU launches a working kernel, cooperate with each other to do ring-based reduction
- A common backend for popular DL frameworks such as PyTorch

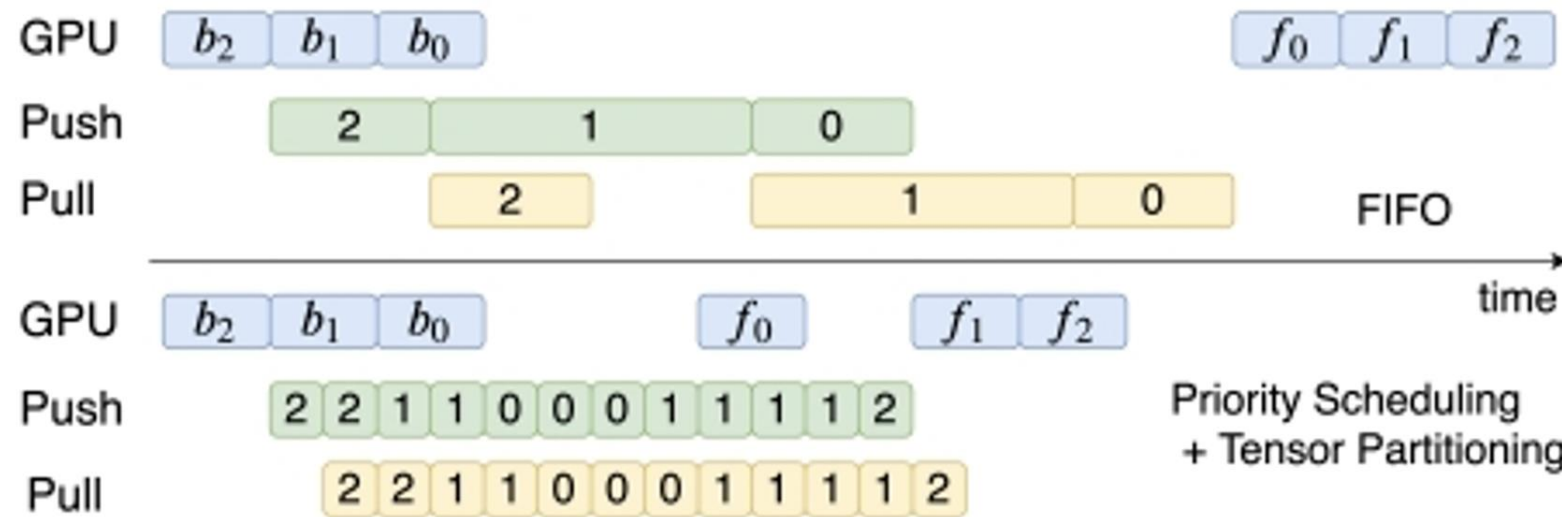
- **AllReduce-based Data Parallelism**

- Workers exchange updates in a ring/tree topology
- Communication traffic: $2N/P * (P - 1)$ (N : data size; P : # of workers)
- Widely-used (e.g., NCCL) yet sensitive to stragglers



Federated Learning
~ PS-based Data Parallelism





Partitioning and reordering communication allows better overlap

LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training

Meta GenAI¹